

Google Developer Groups

On Campus • SATI Vidisha

Workshop On

FOUNDATIONS OF PRACTICAL ENGINEERING AND EXECUTION

WELCOME EVERYONE

FOUNDATION OF PRACTICAL ENGINEERING AND EXECUTION

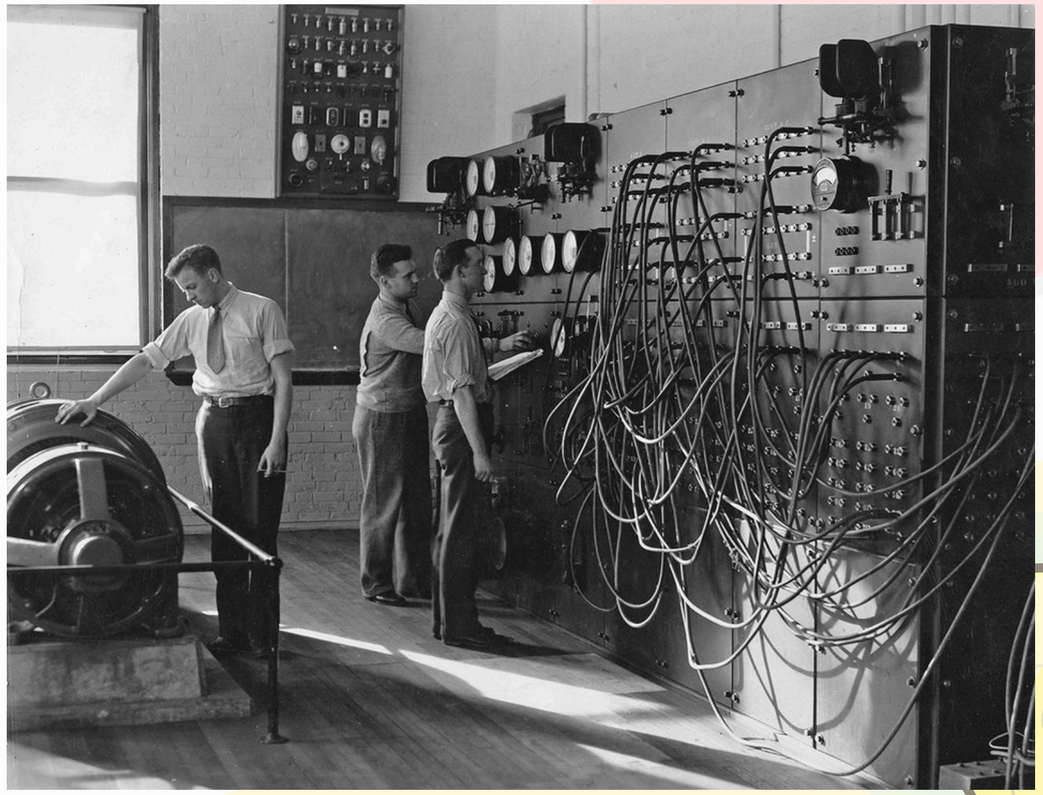
BY NIKHIL KUSHWAH

(6TH SEM, ELECTRICAL ENGINEERING, SATI VIDISHA)

ENROLLMENT NO - 0108EE231009

E- MAIL : nikhilkushwah078@gmail.com

CONTACT - 7987260228



Date - 21/06/2026

Day - Sunday

WORKSHOP OVERVIEW

- **The History of Electricity:** Exploring the key discoveries, the evolution of electrical power, and the foundational principles that shaped modern engineering.

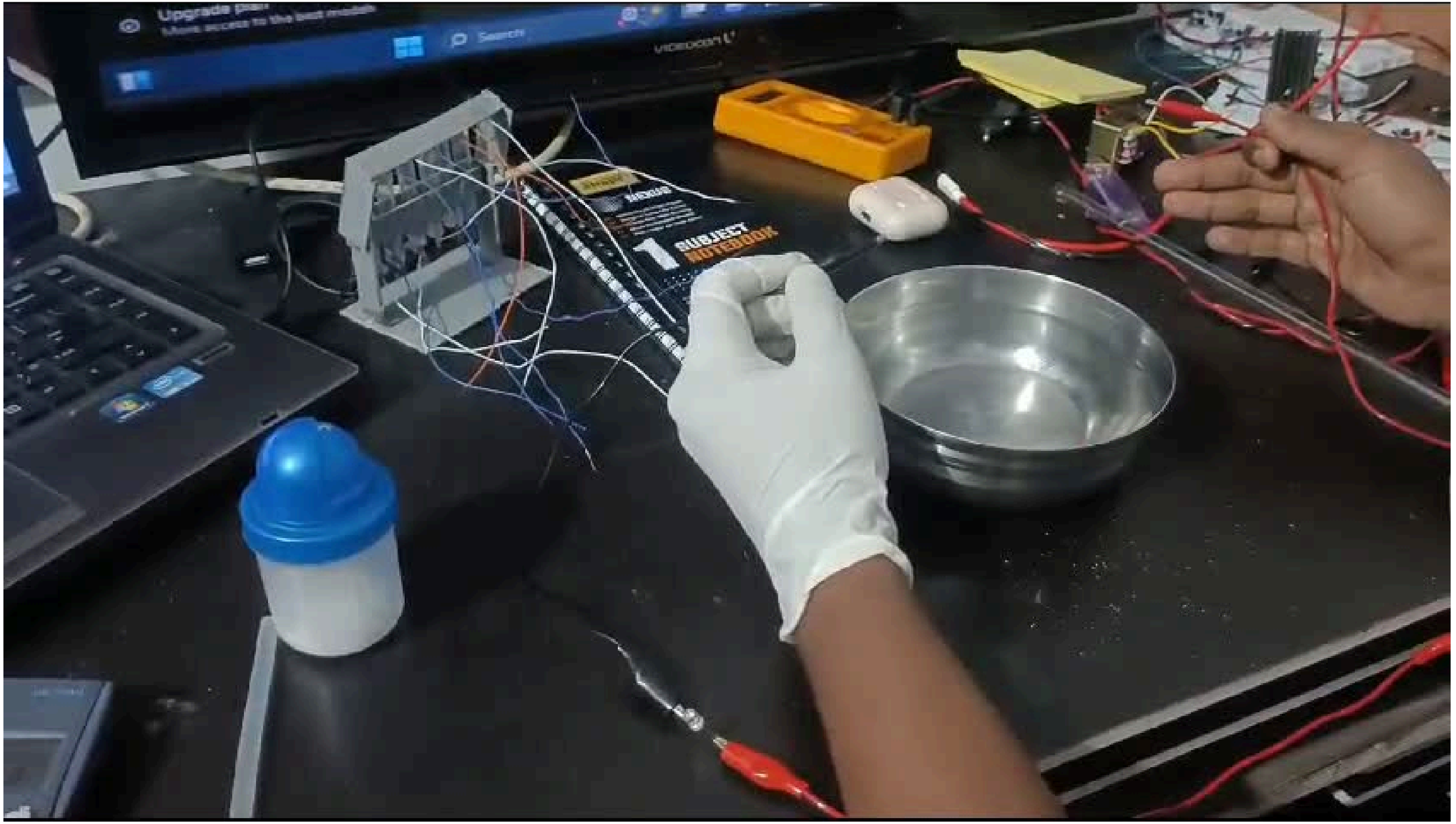
- **Basic Electronic Components & Applications:** A practical look at core hardware elements—like resistors, capacitors, and transistors—and how they operate fundamentally within a circuit.

- **Basics of Logic Building in Programming:** Developing algorithmic thinking, structured problem-solving, and the foundational logic required before writing code.

- **How AI Differs from Humans:** Examining the core distinctions between human intelligence and machine learning, comparing data-driven algorithms to human intuition and cognition.

- **Questions discussion session**


SALT WATER IS A GREAT CONDUCTOR OF
ELECTRICITY. 8TH GRADE PHYSICS


Upgrade plan
Search
SUBJECT
NOTEBOOK

OBJECTIVE OF THIS WORKSHOP

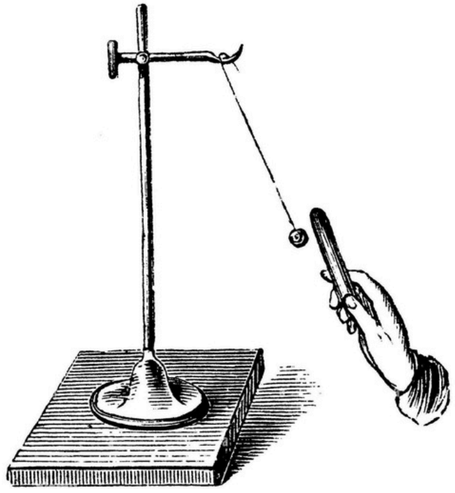
The objective of this workshop is to introduce participants to the practical applications and hands-on implementations of core engineering and foundational electronic concepts. Furthermore, the session aims to familiarize attendees with the fundamental principles of modern engineering, bridging the gap between textbook theory and real-world technological innovation

What is electricity

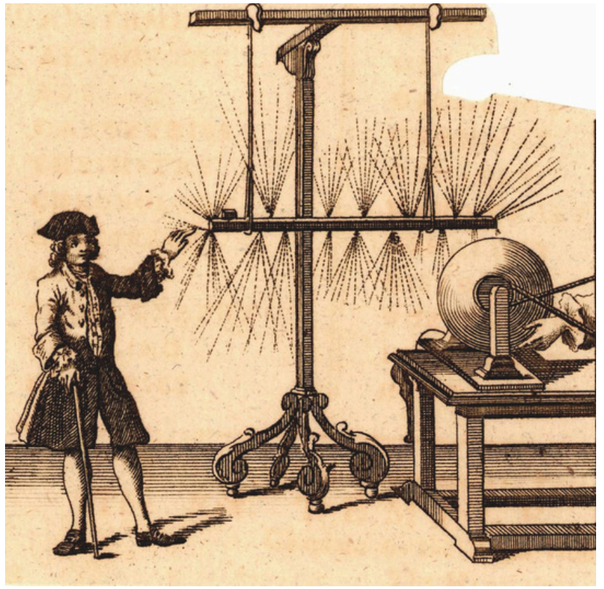


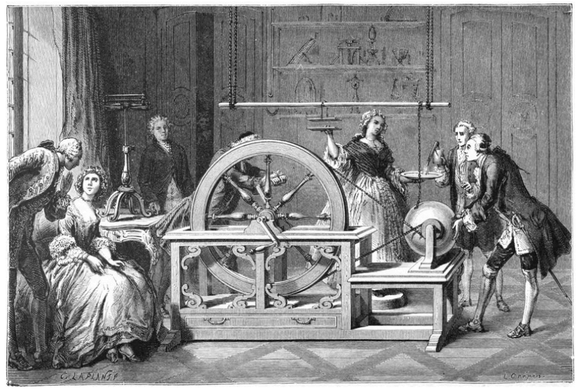
History of Electricity(Static Electricity)

- 600 BC (Thales of Miletus, Greece): Rubbing amber with fur produced static charges → earliest record of electricity.
- 1600 (William Gilbert, England): Coined the term “electricus”, distinguished magnetic and electric phenomena.
- 1660 (Otto von Guericke, Germany): Built the first electrostatic generator (sulfur globe).
- 1705 (Francis Hauksbee, England): Created the electrostatic machine that produced sparks and glowing effects.



Static Electricity

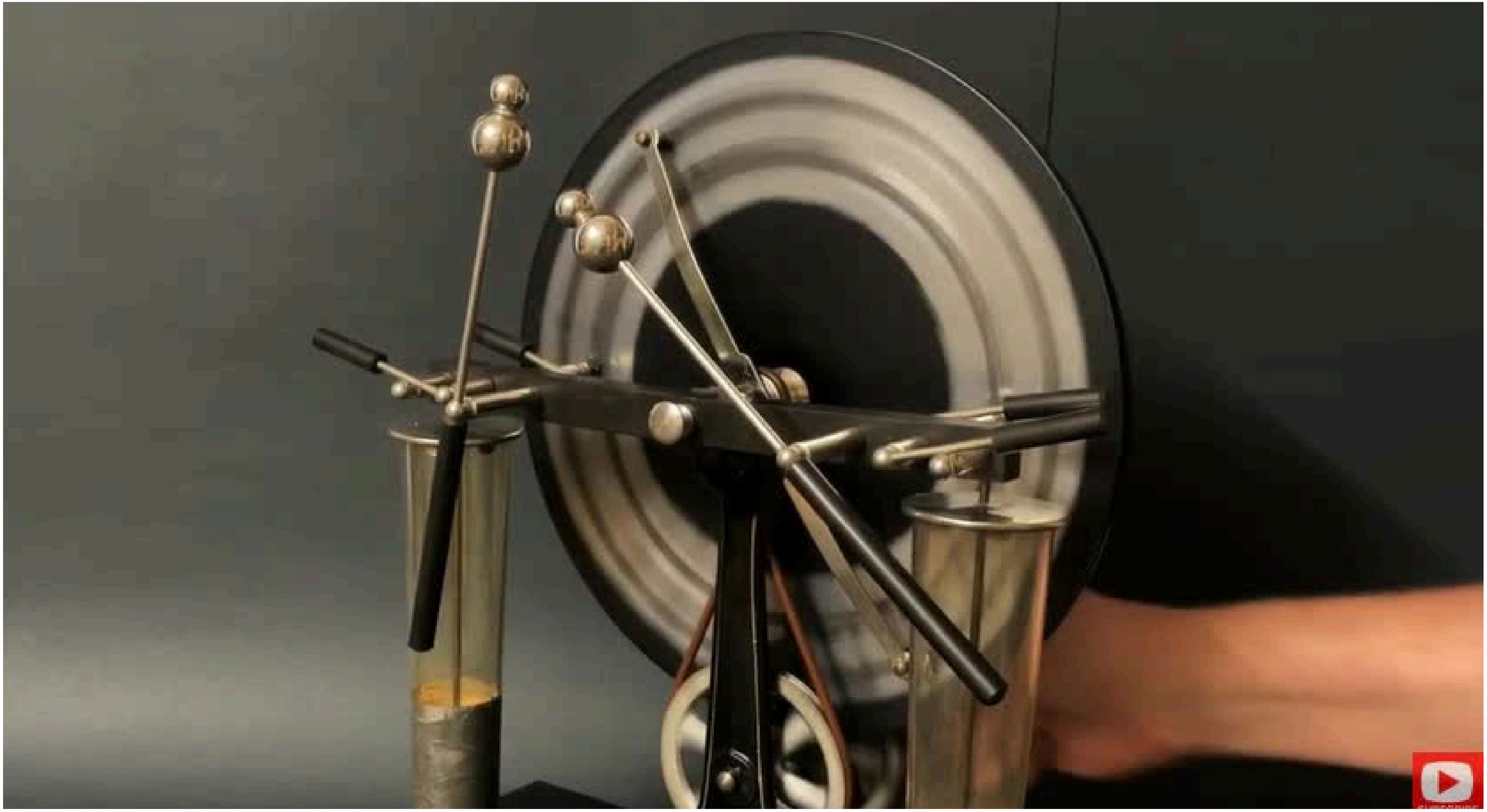




Francis hauksbee spark generator

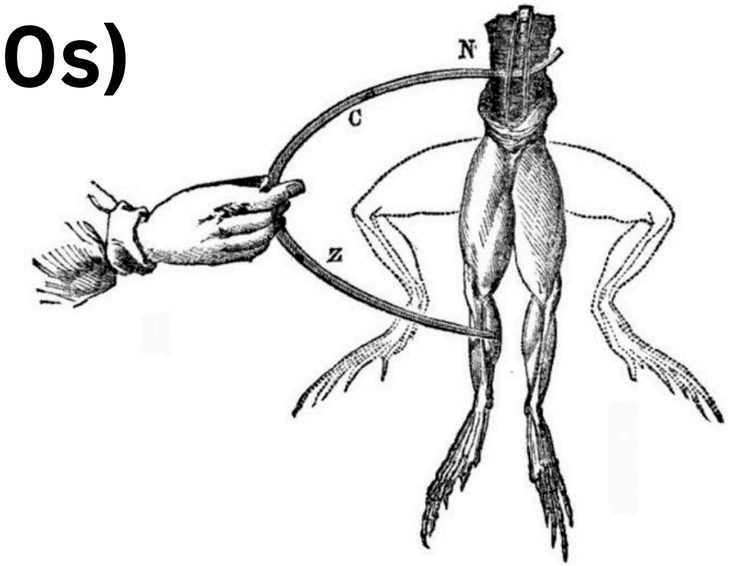


otta von static electricity genrator



Discovery Of Continues Electricity

Luigi Galvani’s Frog Leg Experiment (1780s)

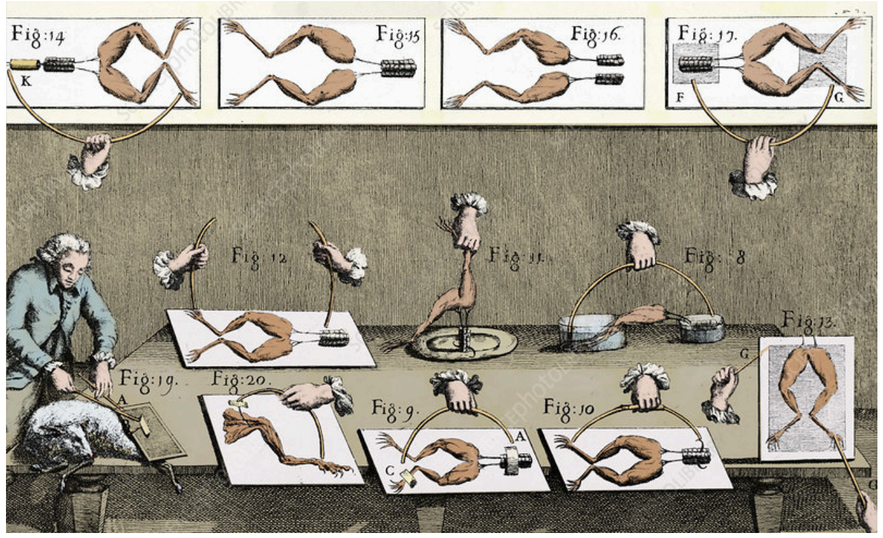


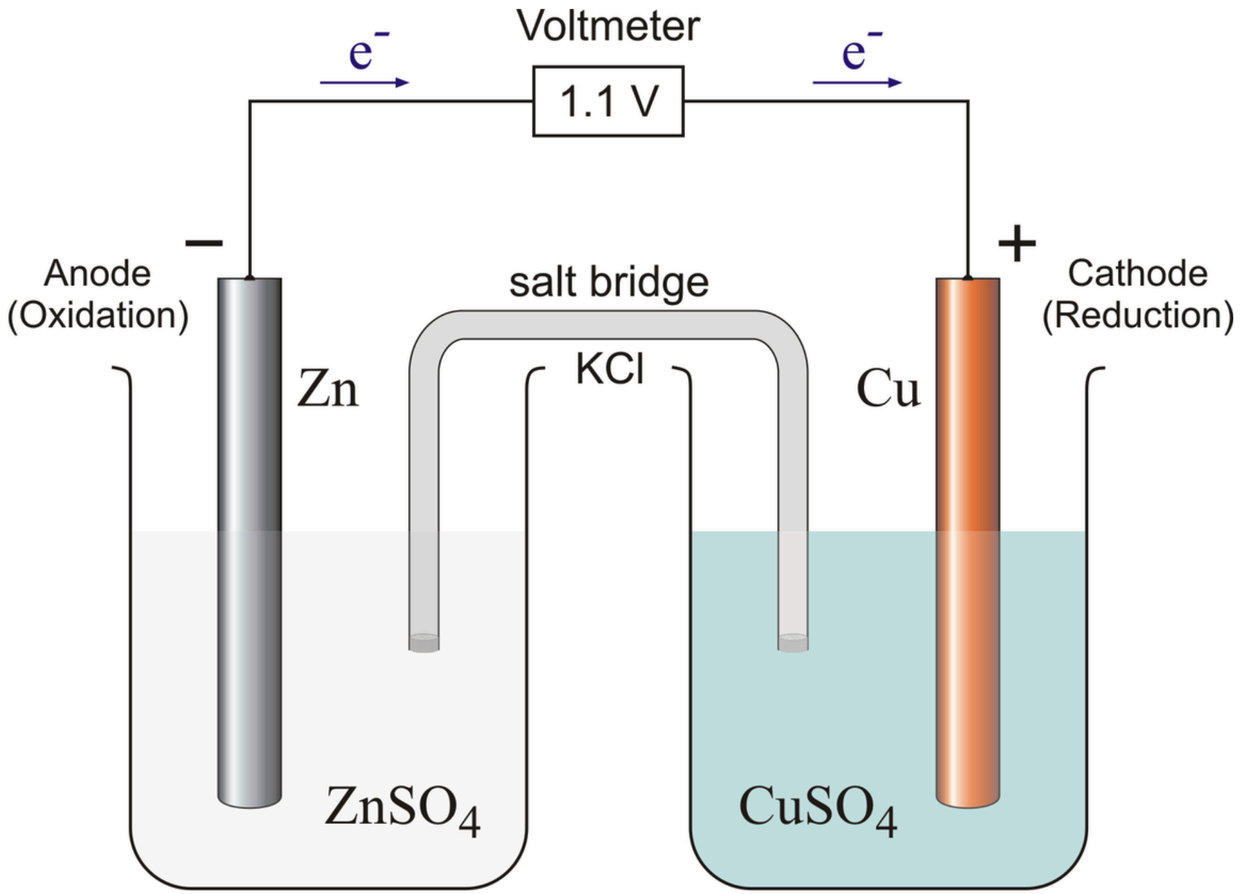
- Luigi Galvani (Italian scientist & physician) was studying anatomy in the 1780s.
- During one experiment, he noticed that when a frog’s leg muscle was touched with a metal scalpel while near an electrostatic machine, the leg twitched as if alive.

- He believed he had discovered “animal electricity” → a vital life force in the frog’s nerves and muscles.
- Thought living beings generated electricity inside their bodies.

- In reality, Galvani had accidentally discovered the principle of a battery, but he didn’t realize it.

- Volta built the Voltaic Pile (1800) → world’s first true battery, producing continuous electricity.





$Zn(s) + 2e^- \longrightarrow Zn^{2+}$ $\quad$ $Cu^{2+} + 2e^- \longrightarrow Cu(s)$

$$E^\circ = E^\circ_{ox} + E^\circ_{red}$$

0.76 V + 0.34 V = 1.1 V

GALVANIC CELL

Early Applications of Continuous Electricity

1. Electrochemistry – Humphry Davy (1807–1808)

- Showed that electricity can break chemical bonds → foundation of electrochemistry.

2. Electromagnetism – Hans Christian Ørsted (1820)

- Discovered that a current-carrying wire deflects a magnetic compass needle.

3. Electromagnetic Induction – Michael Faraday (1831)

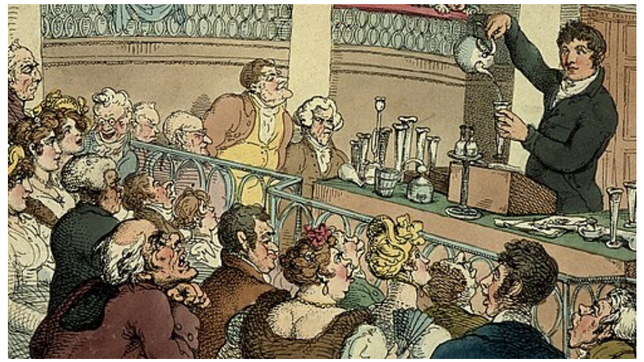
- Showed that moving a magnet through a coil produces an electric current.

4. Telegraphy – Joseph Henry & Samuel Morse (1830s–1840s)

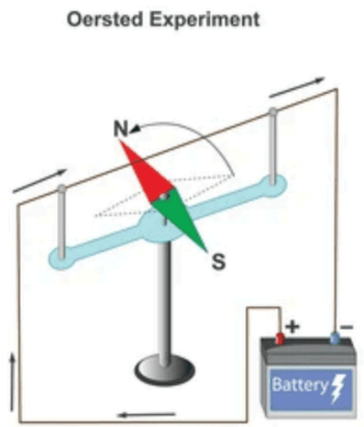
- Samuel Morse used this to invent the electric telegraph and Morse Code.

5. Mathematical Foundation – Georg Ohm (1827)

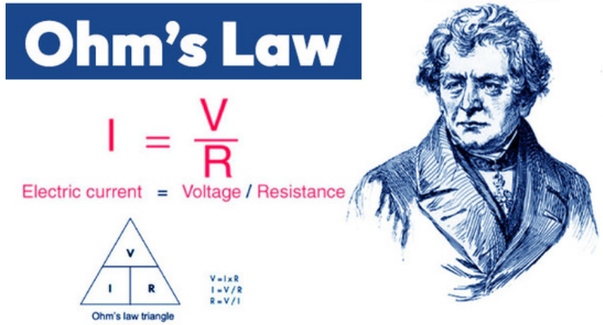
- Discovered the relationship **V= IR.**

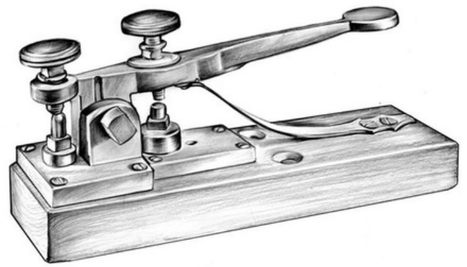




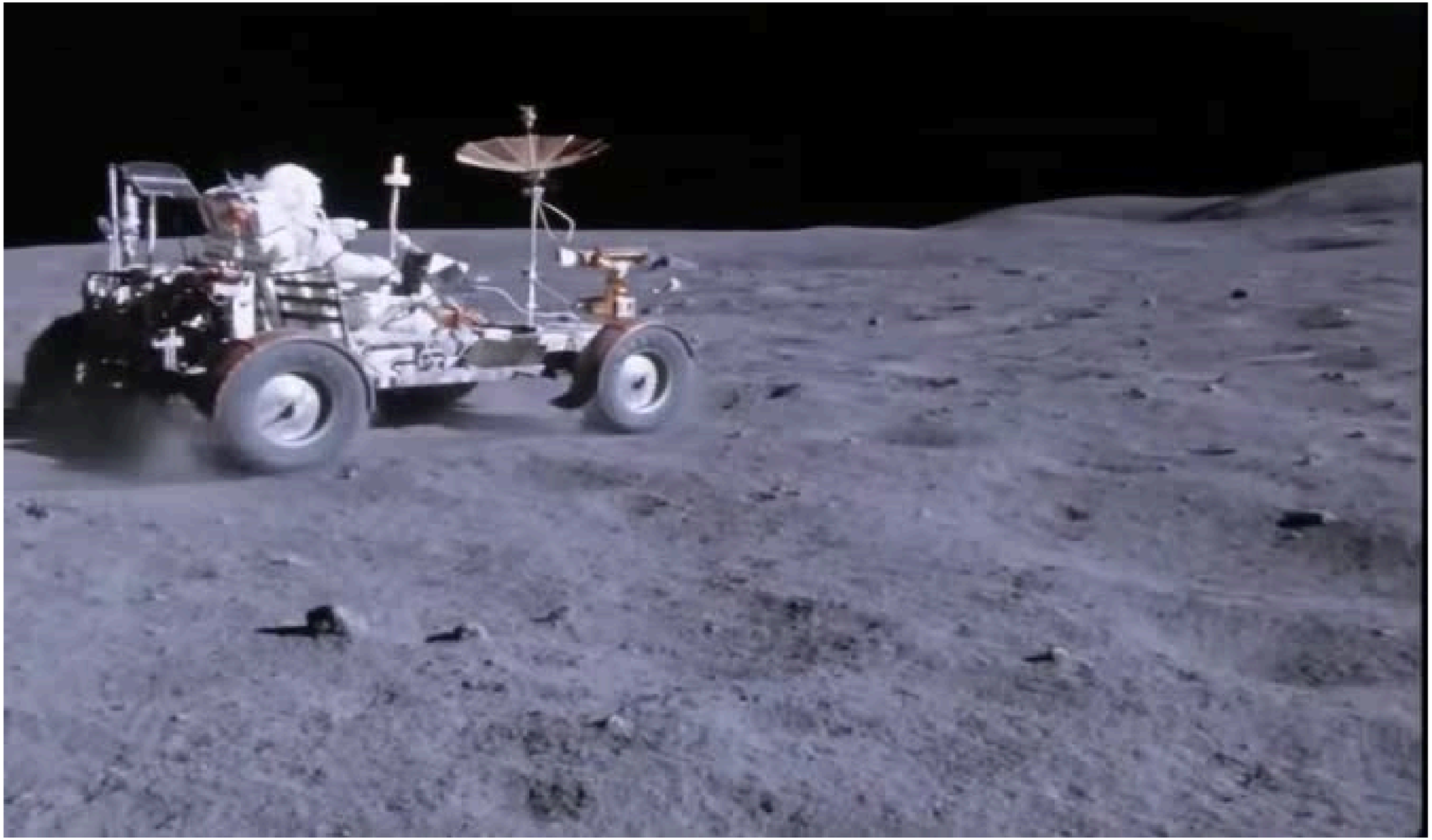


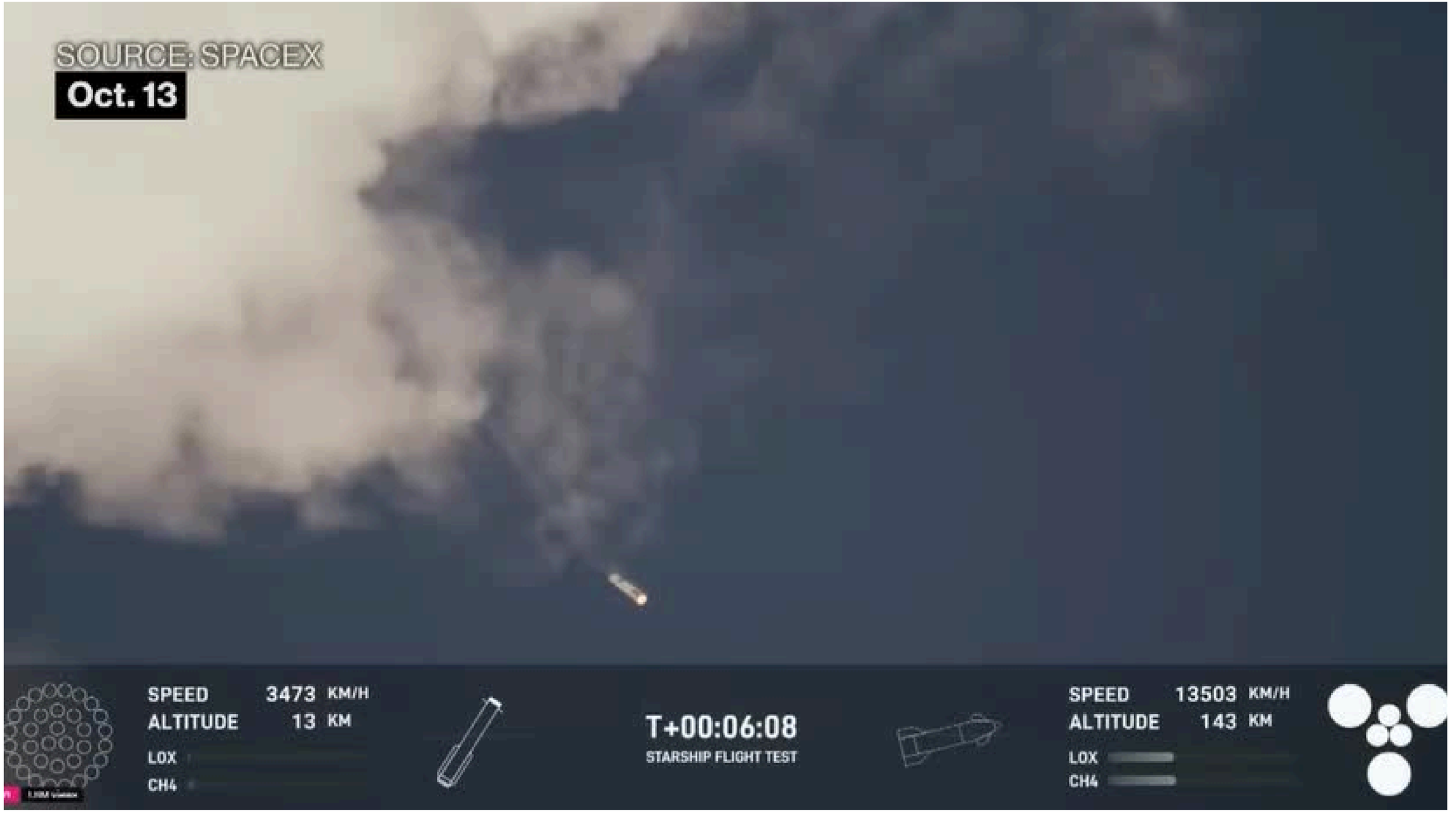






HUGE TECHNICAL ADVANCEMENT IN 20TH CENTURY




SOURCE: SPACEX
Oct. 13
SPEED 3473 KM/H
ALTITUDE 13 KM
LOX
CH4
T+00:06:08
STARSHIP FLIGHT TEST
SPEED 13503 KM/H
ALTITUDE 143 KM
LOX
CH4

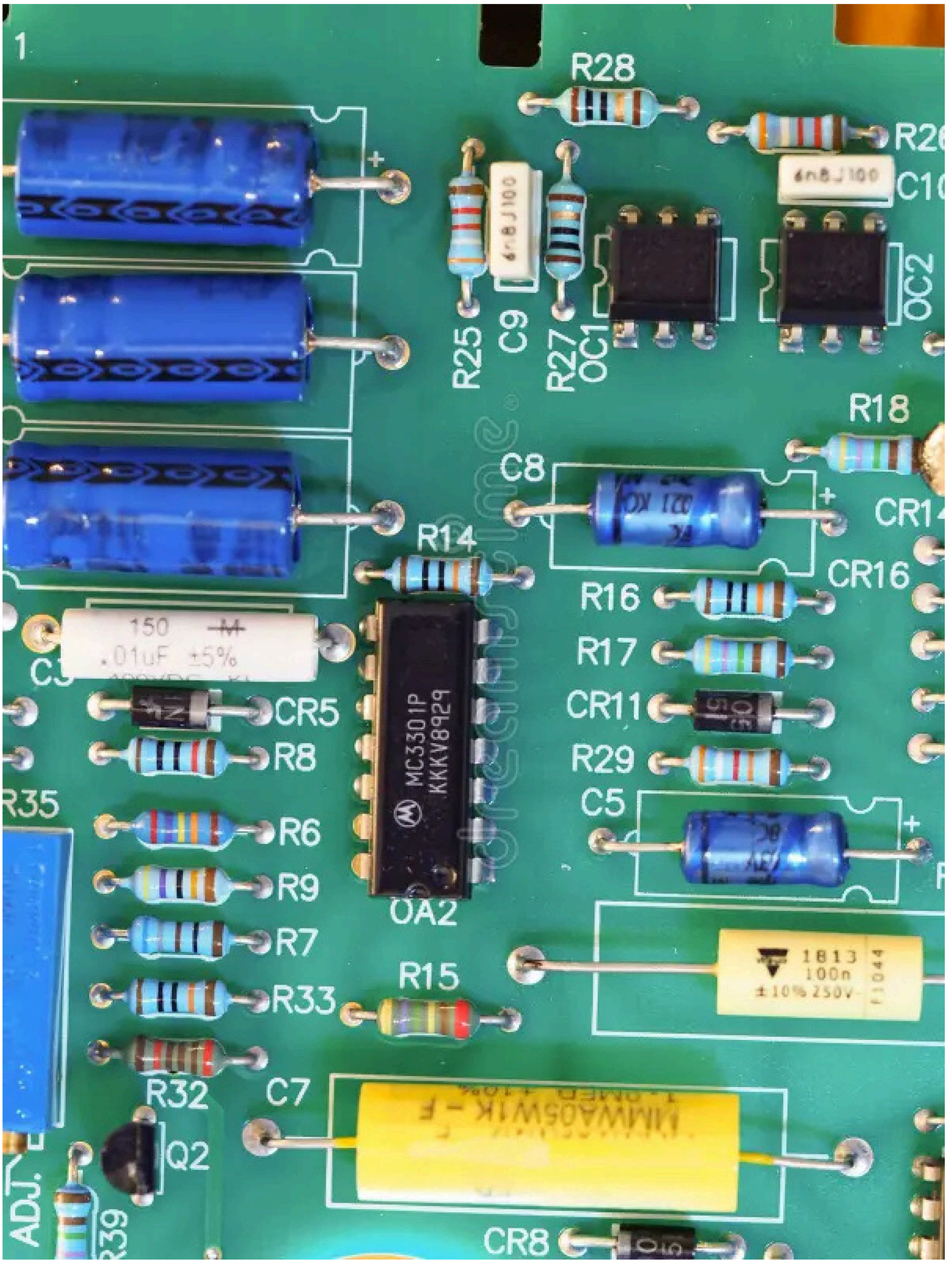
RESISTORS

WHAT IS A RESISTOR

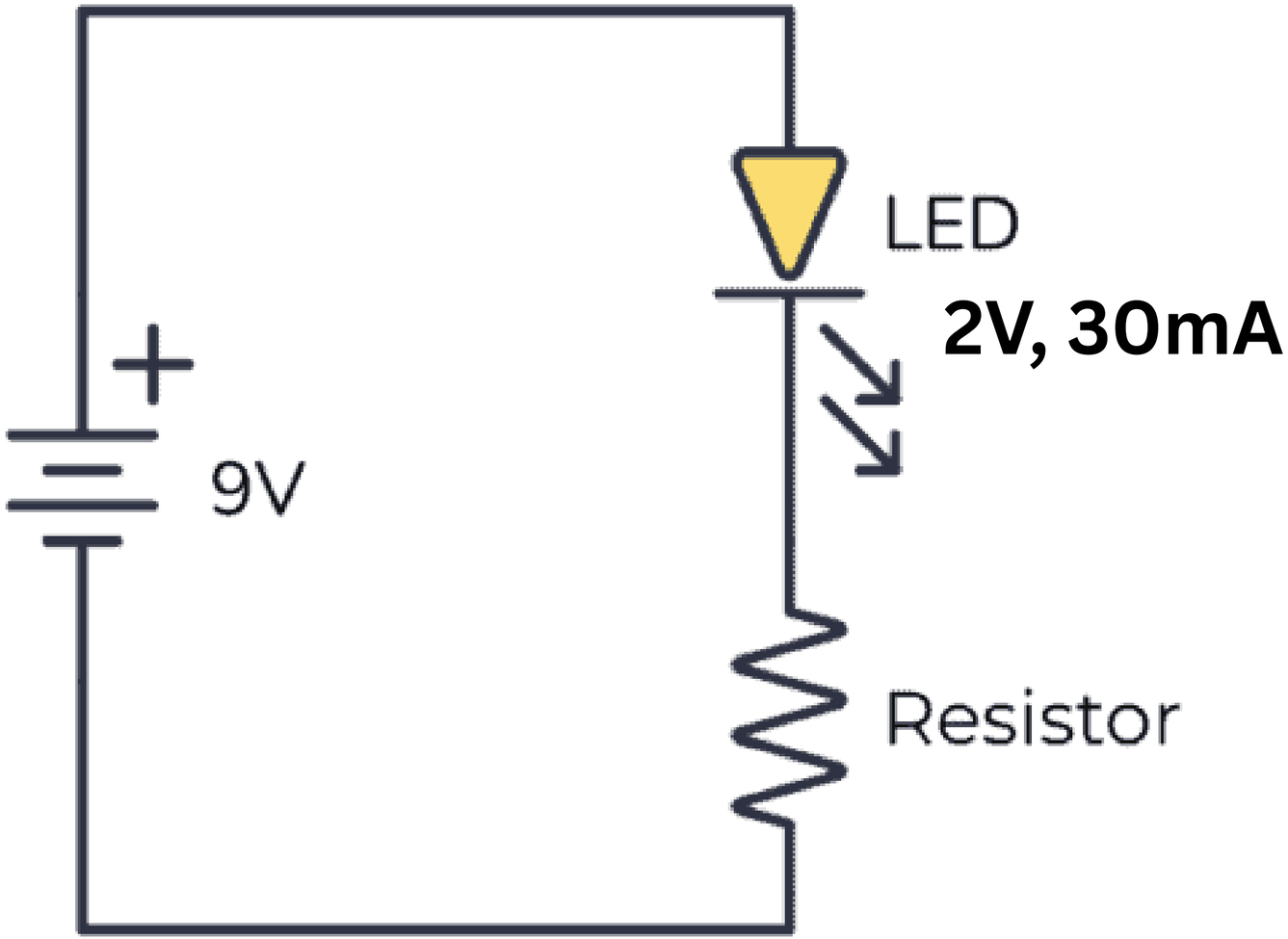
A resistor is an electrical component that opposes or resists the flow of electric current in a circuit. It is used to control the amount of current, divide voltages, and protect other components from excessive current.






1
R28
R25
C9
R27
OC1
OC2
R18
C8
CR14
R14
CR16
R16
R17
CR11
R29
C5
CR5
R8
R35
R6
R9
R7
R33
R15
OA2
MC3301P
KKKV8929
R32
C7
Q2
CR8
ADJ.

SIMPLE QUESTION



Step 2: Calculate R

$$R = \frac{V}{I}$$

$$R_{Resistor} = \frac{V_R}{I_{LED}}$$

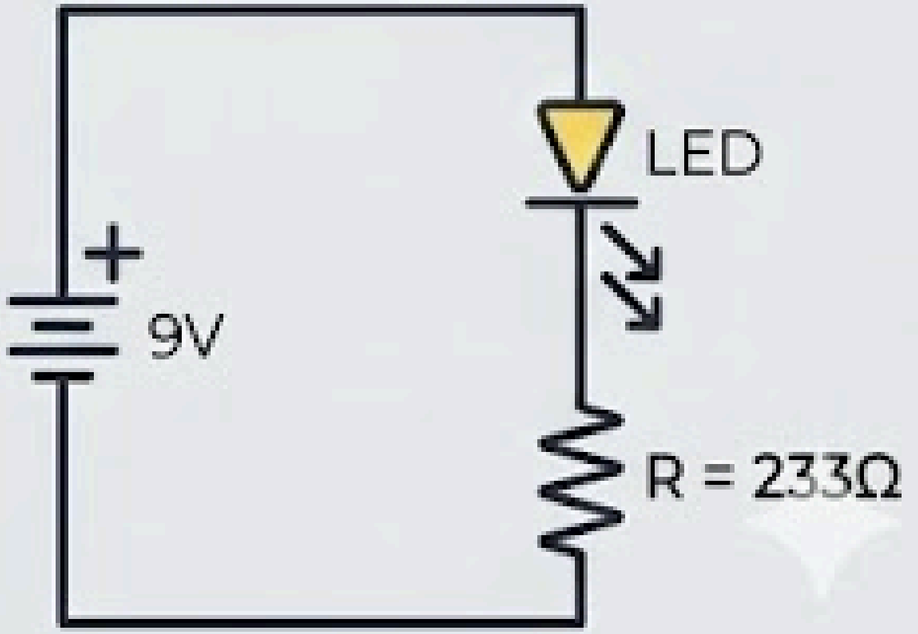
$$R_{Resistor} = \frac{7V}{30mA} \leftarrow (30mA = 0.03A)$$

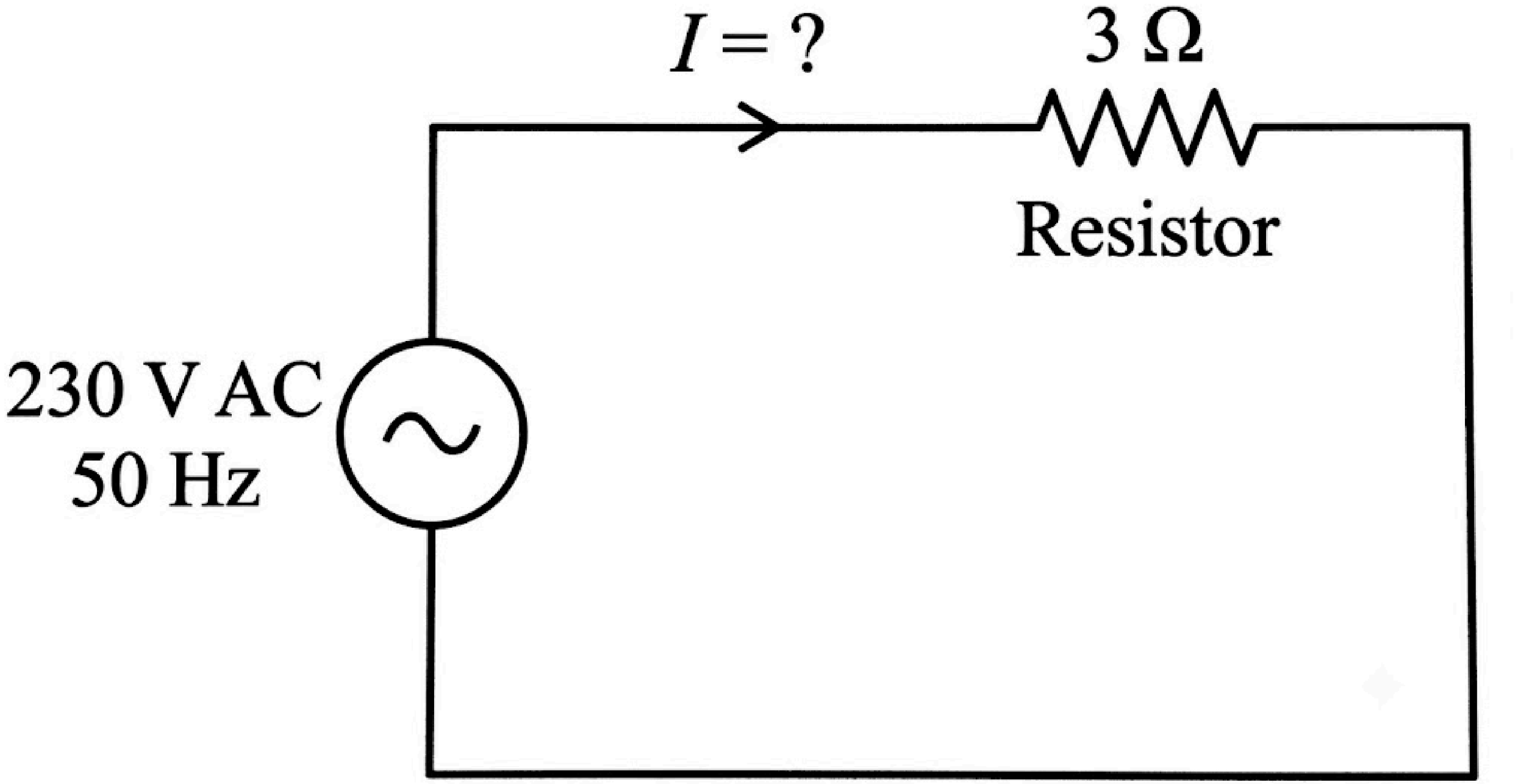
$$R_{Resistor} = \frac{7}{0.03} = 233.3 \text{ Ohms}$$

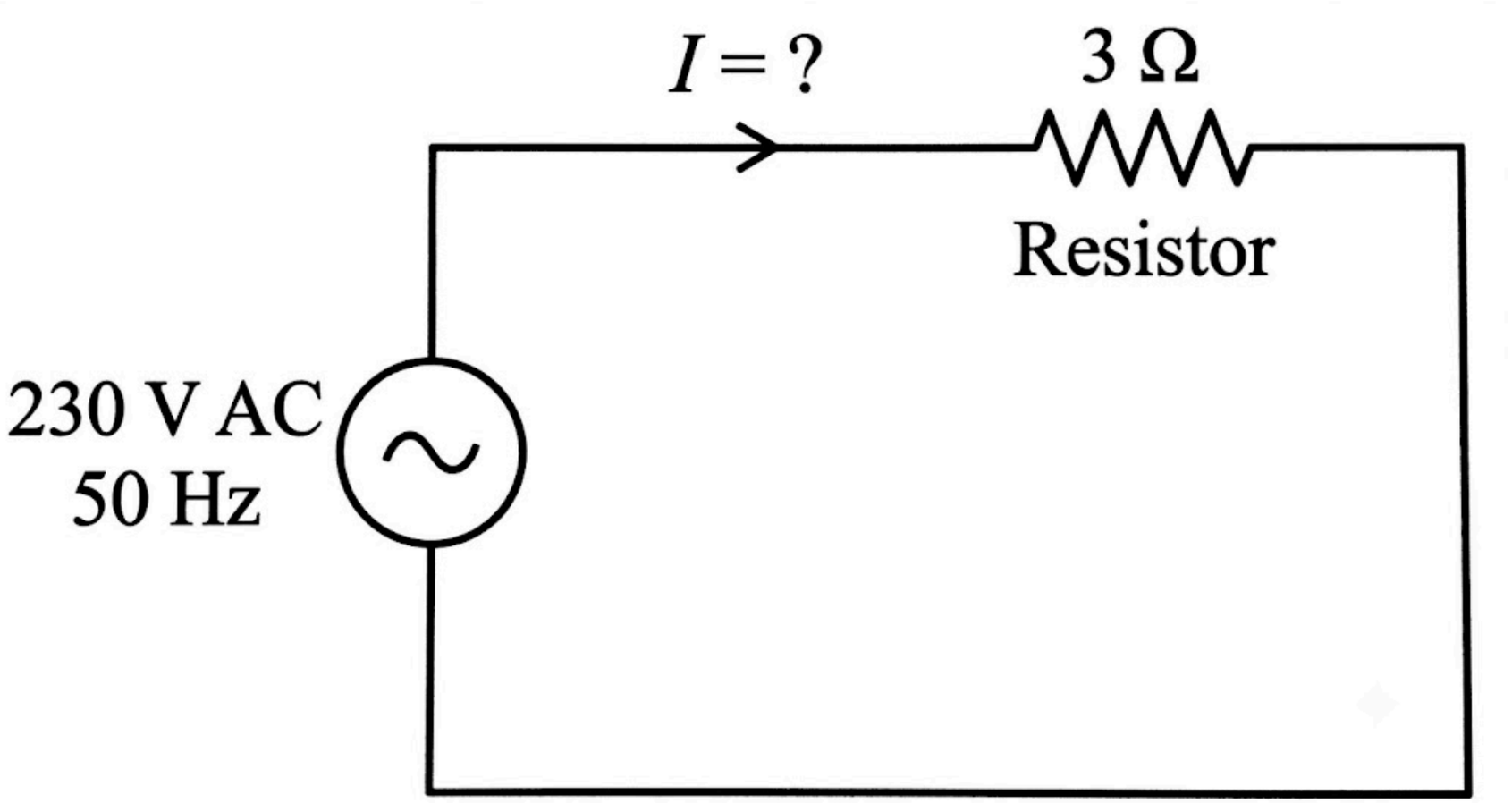
Final Answer and Summary

The required Resistor value is approximately 233Ω (or 233.3Ω)






$I = ?$
3 Ω
Resistor
230 V AC
50 Hz



$$V = IR$$

$$I = V/R$$

$$I = 230/3$$

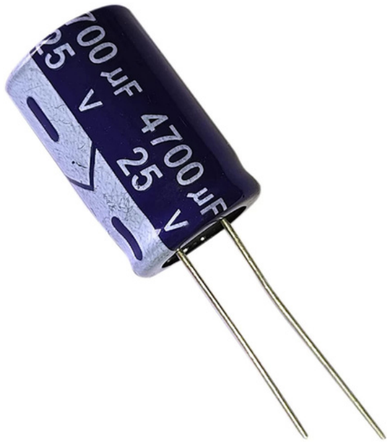
$$\boxed{I = 76.66}$$



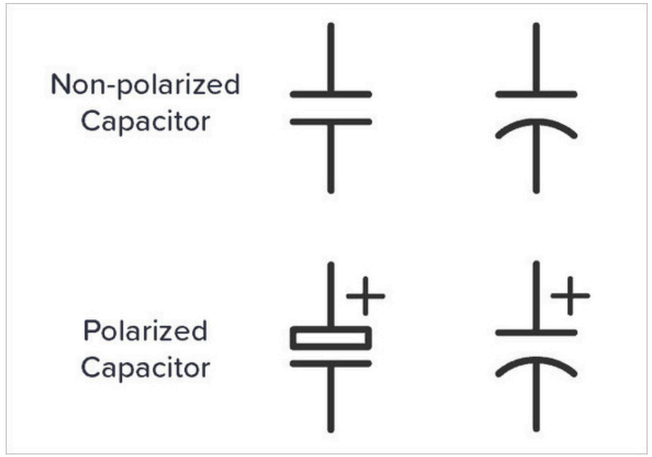
CAPACITOR

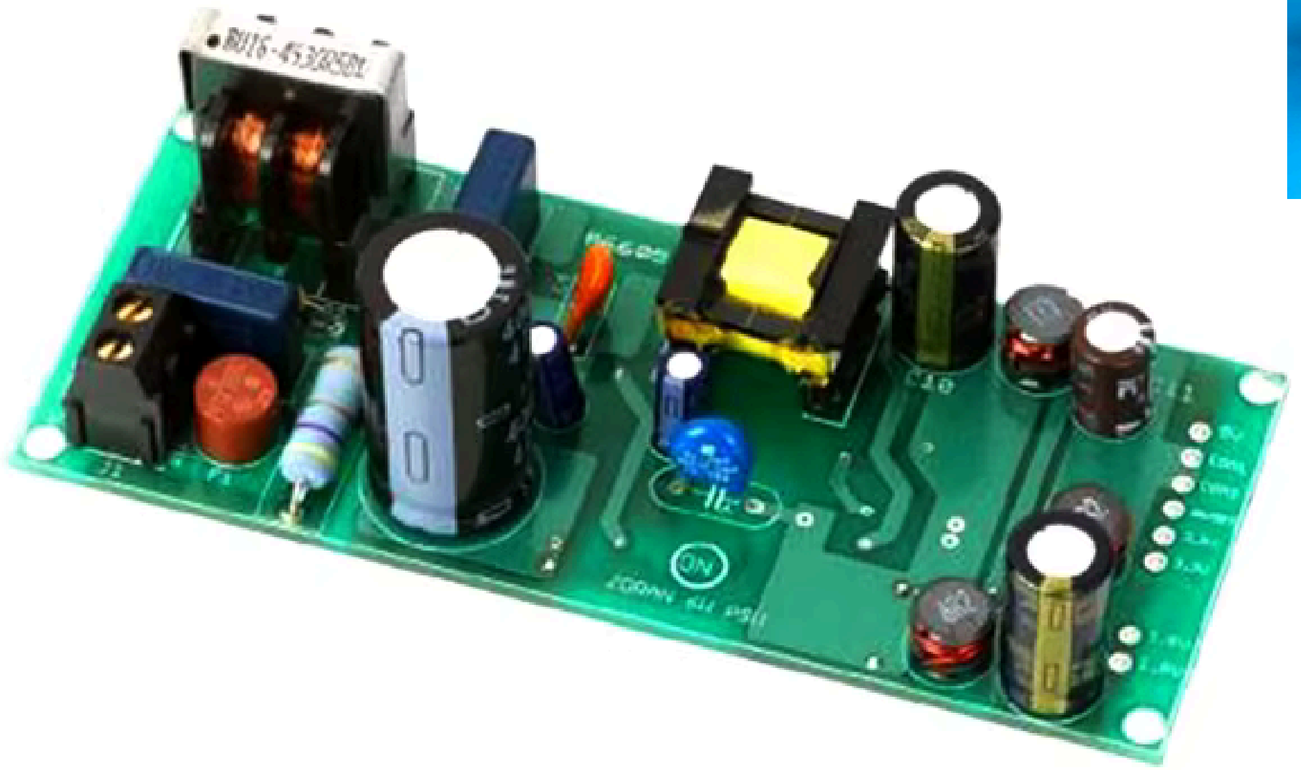
WHAT IS A CAPACITOR

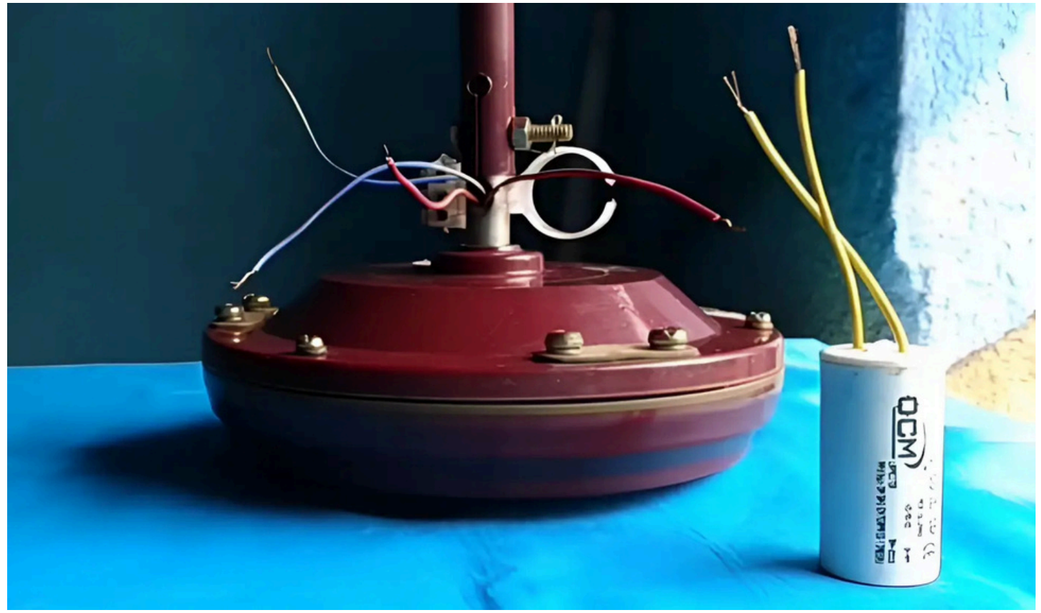
A capacitor is an electrical component that stores electric charge and energy in an electric field between two conductive plates separated by an insulating material called a dielectric.

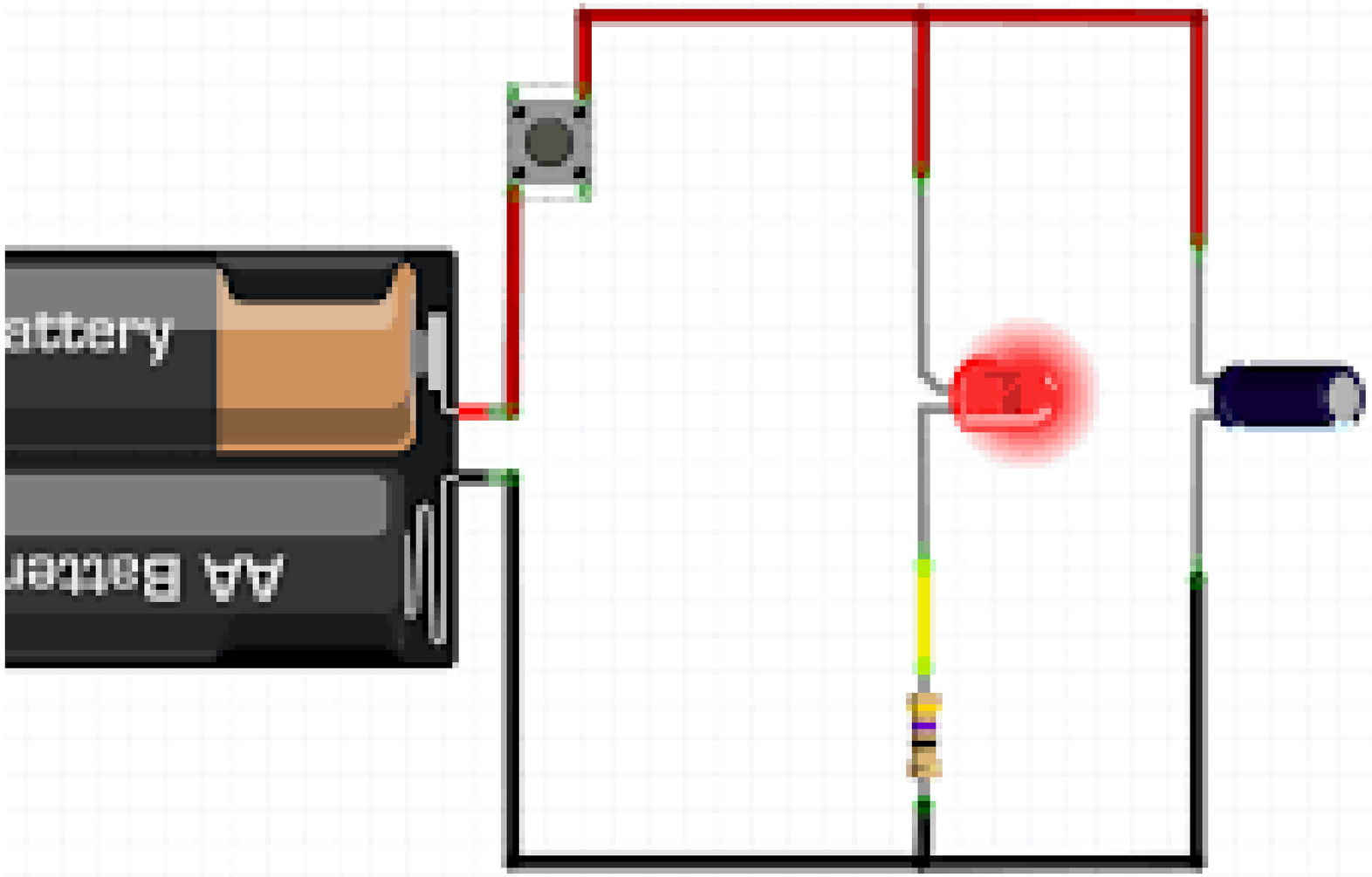




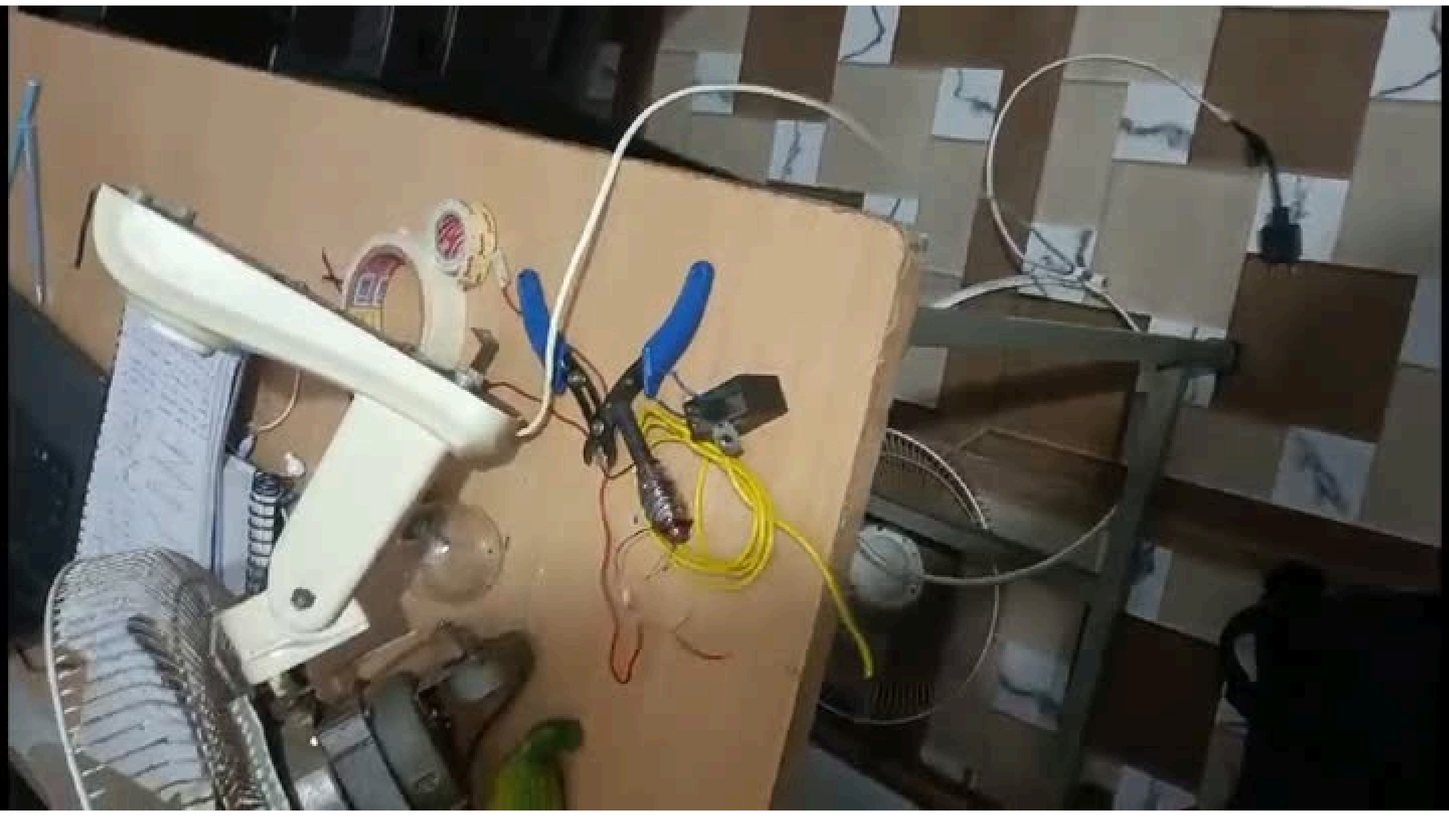








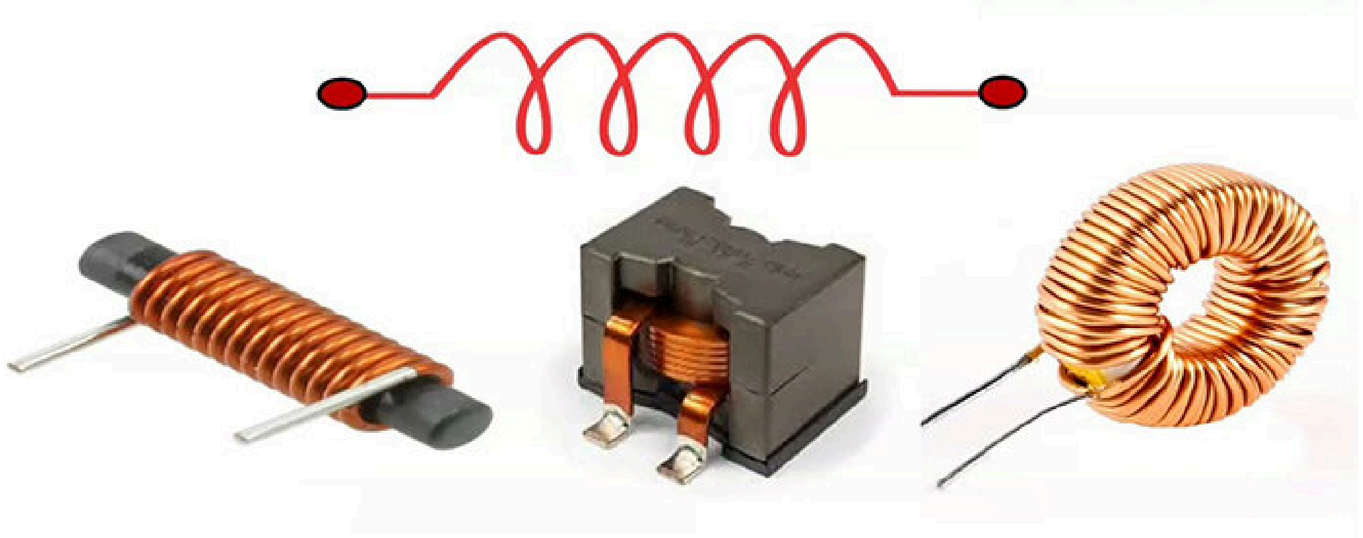
fritzing



INDUCTORS

WHAT IS INDUCTOR

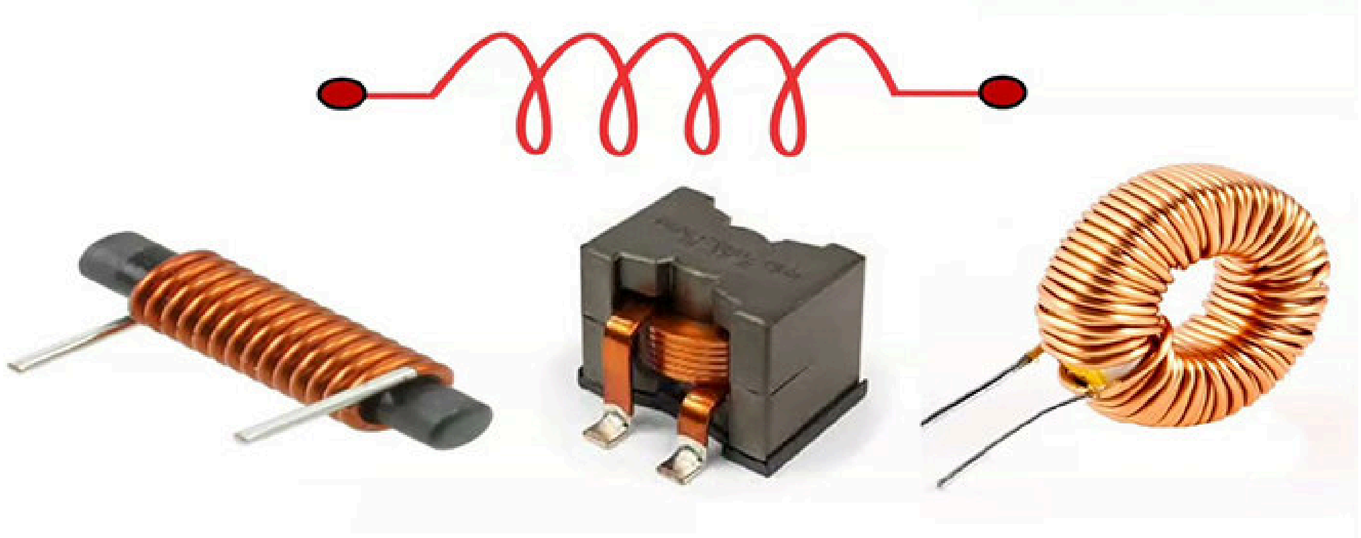
An inductor is a passive, two-terminal electrical component that stores energy in a magnetic field when an electric current flows through it.



INDUCTORS

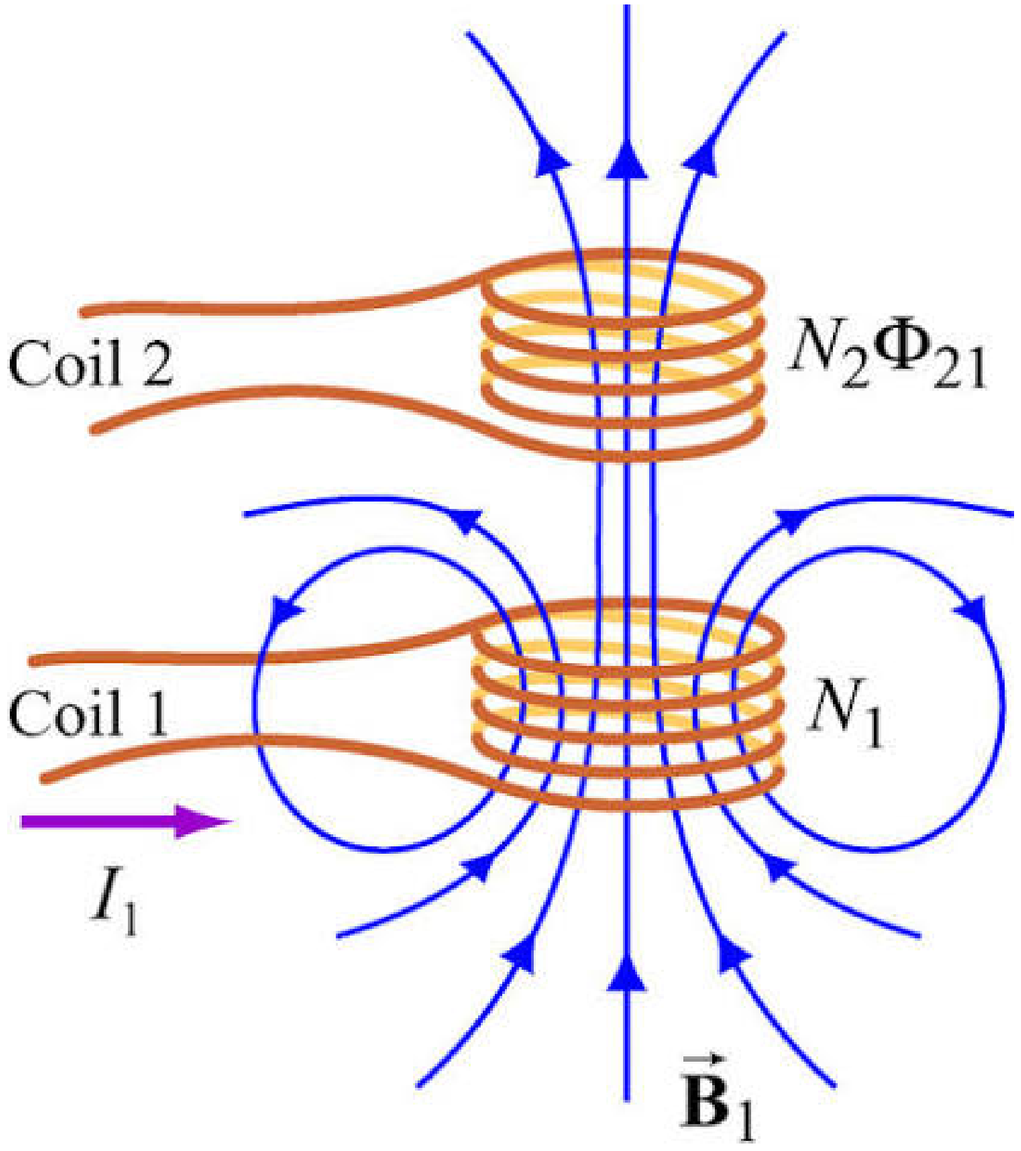
WHAT IS INDUCTOR

An inductor is a passive, two-terminal electrical component that stores energy in a magnetic field when an electric current flows through it.



Mutual Inductance

Mutual Induction is the phenomenon in which a changing electric current in one coil induces an electromotive force (emf) or voltage across a neighboring, magnetically linked coil.

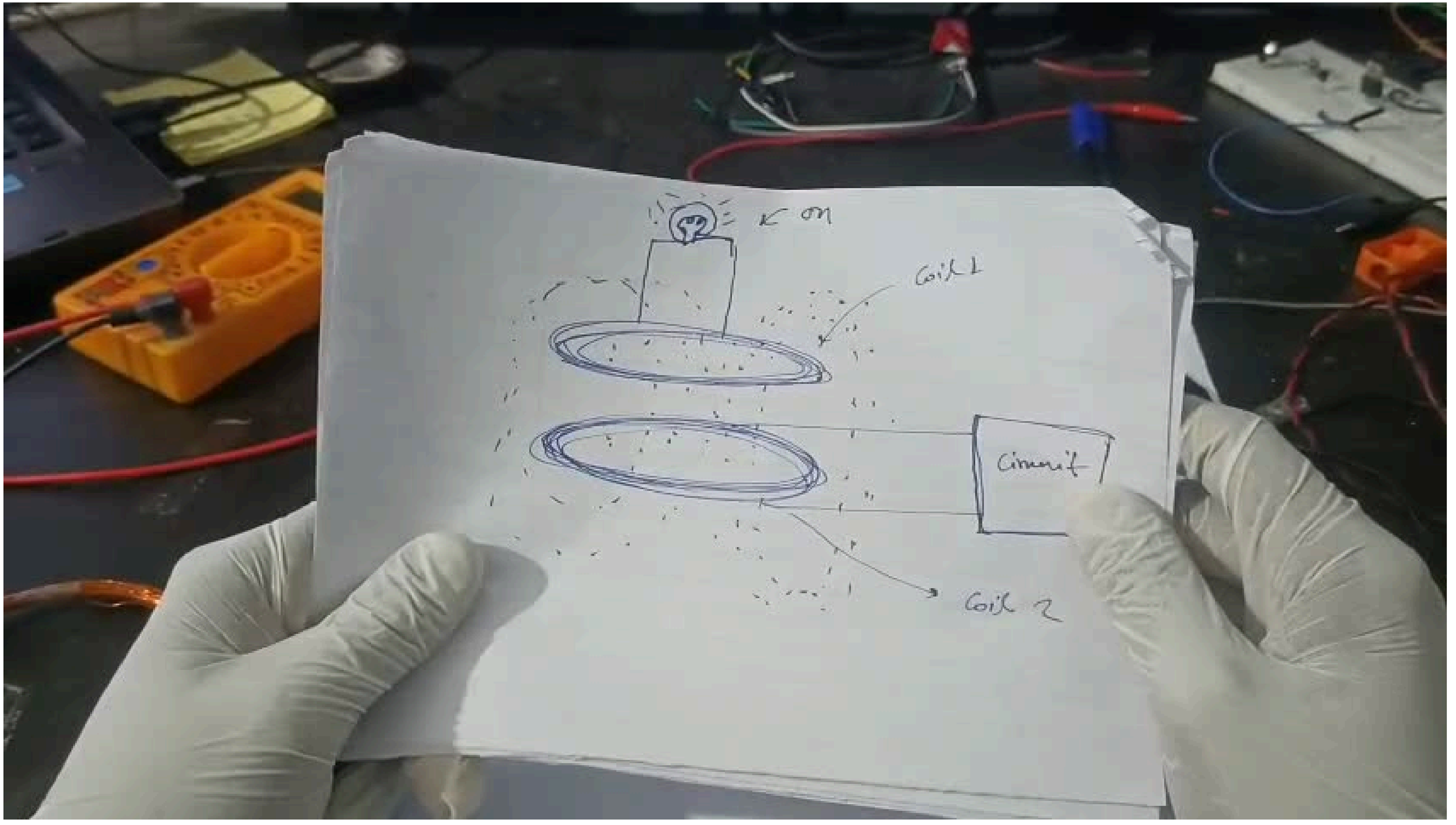




Case 1



Case 2


LED ON
Coil 1
Circuit
Coil 2

JOIN ME ON



https://www.linkedin.com/in/nikhil-kushwah-



https://www.youtube.com/@Nikhilkushwah078

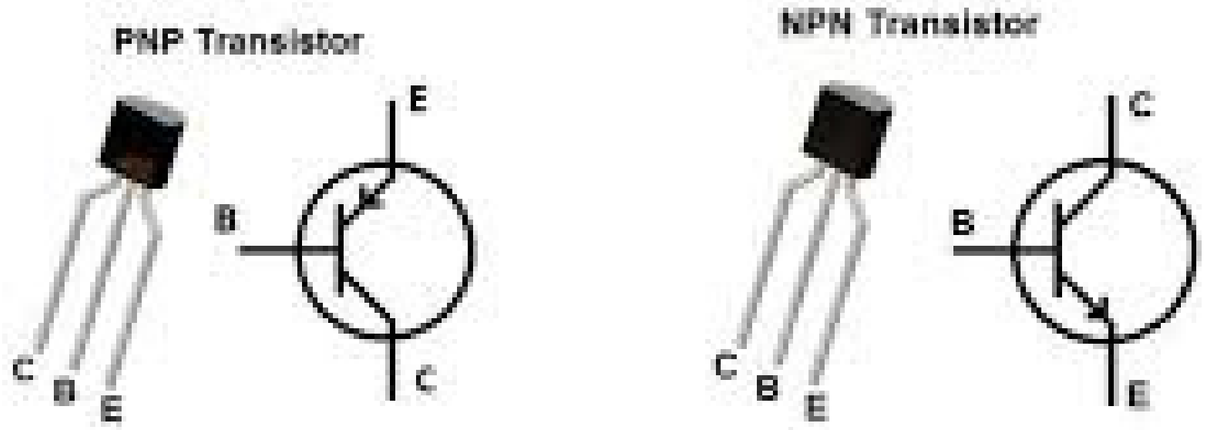


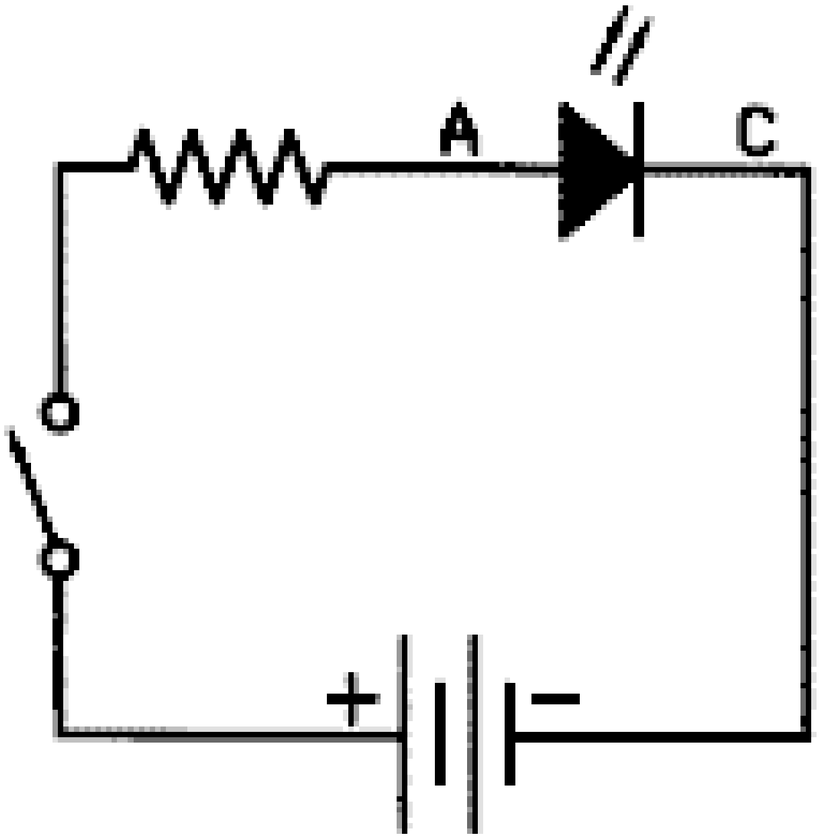
https://github.com/nikhilkush078

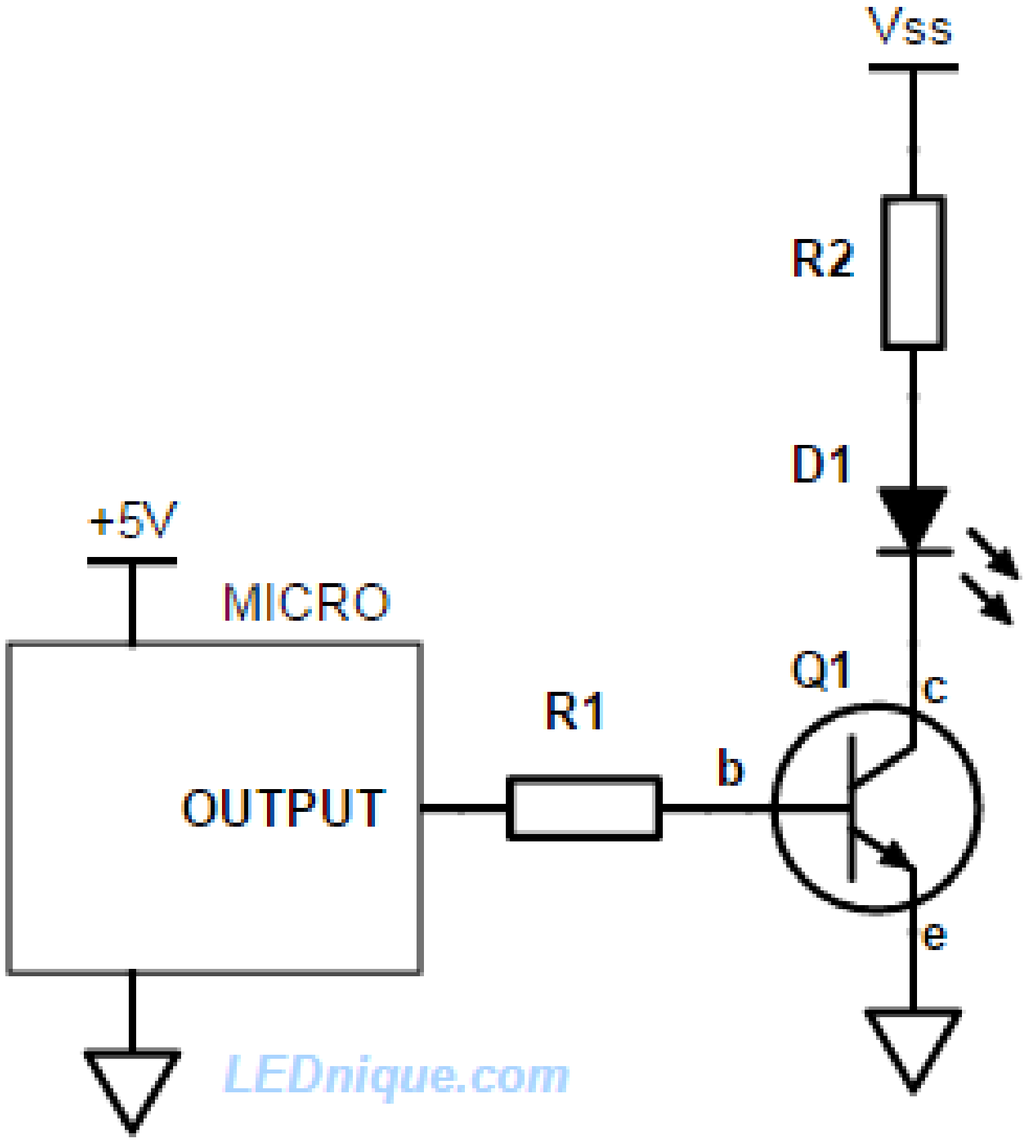
TRANSISTORS

WHAT IS TRANSISTOR

A transistor is an active electronic component that can control current flow, working either as an amplifier or as a switch in electronic circuits.

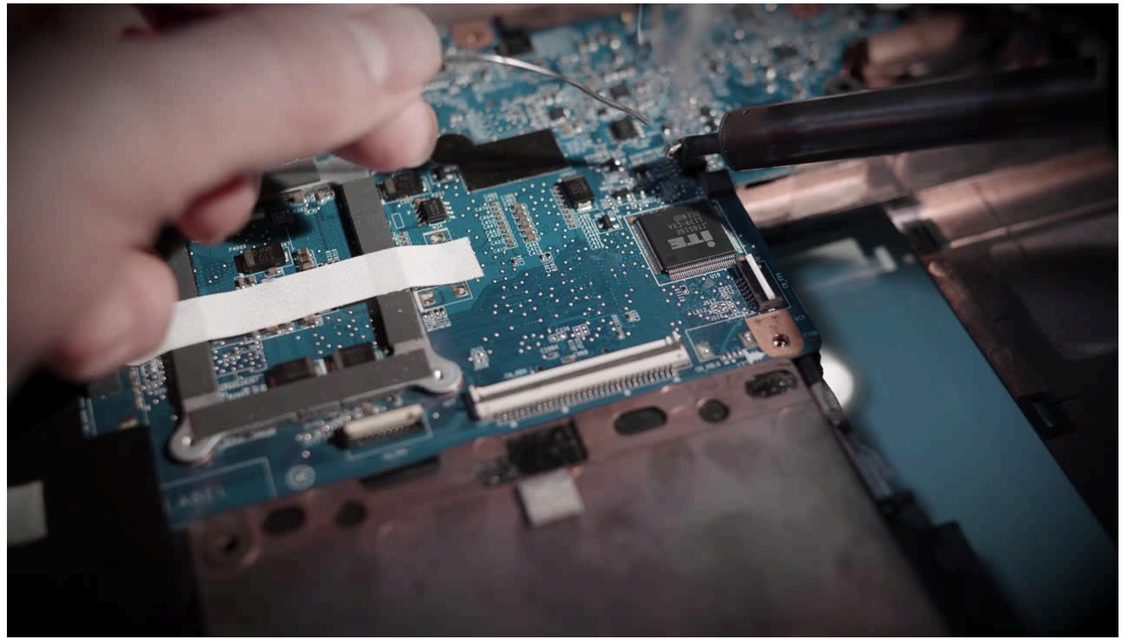


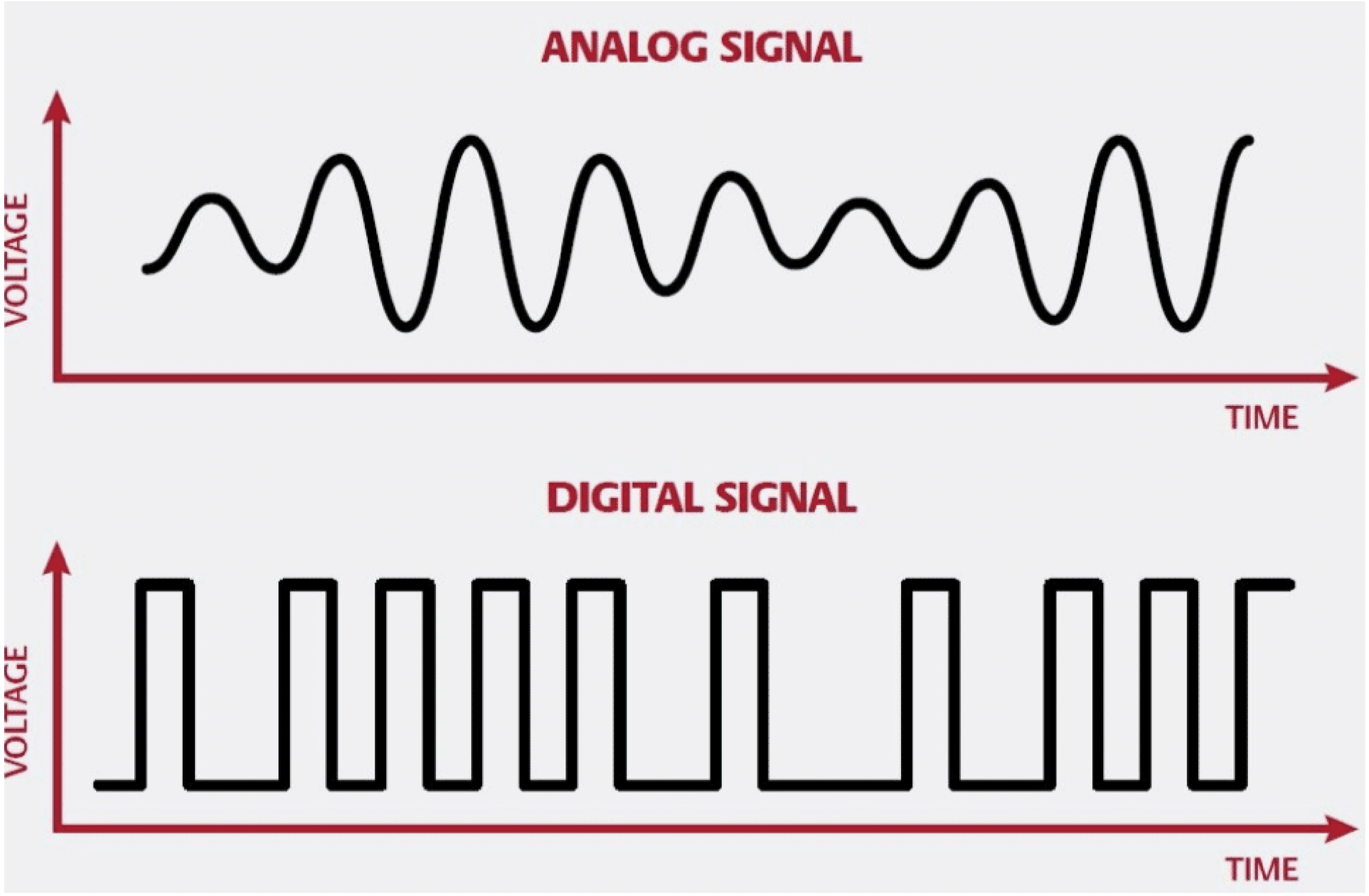

A
C
+
−

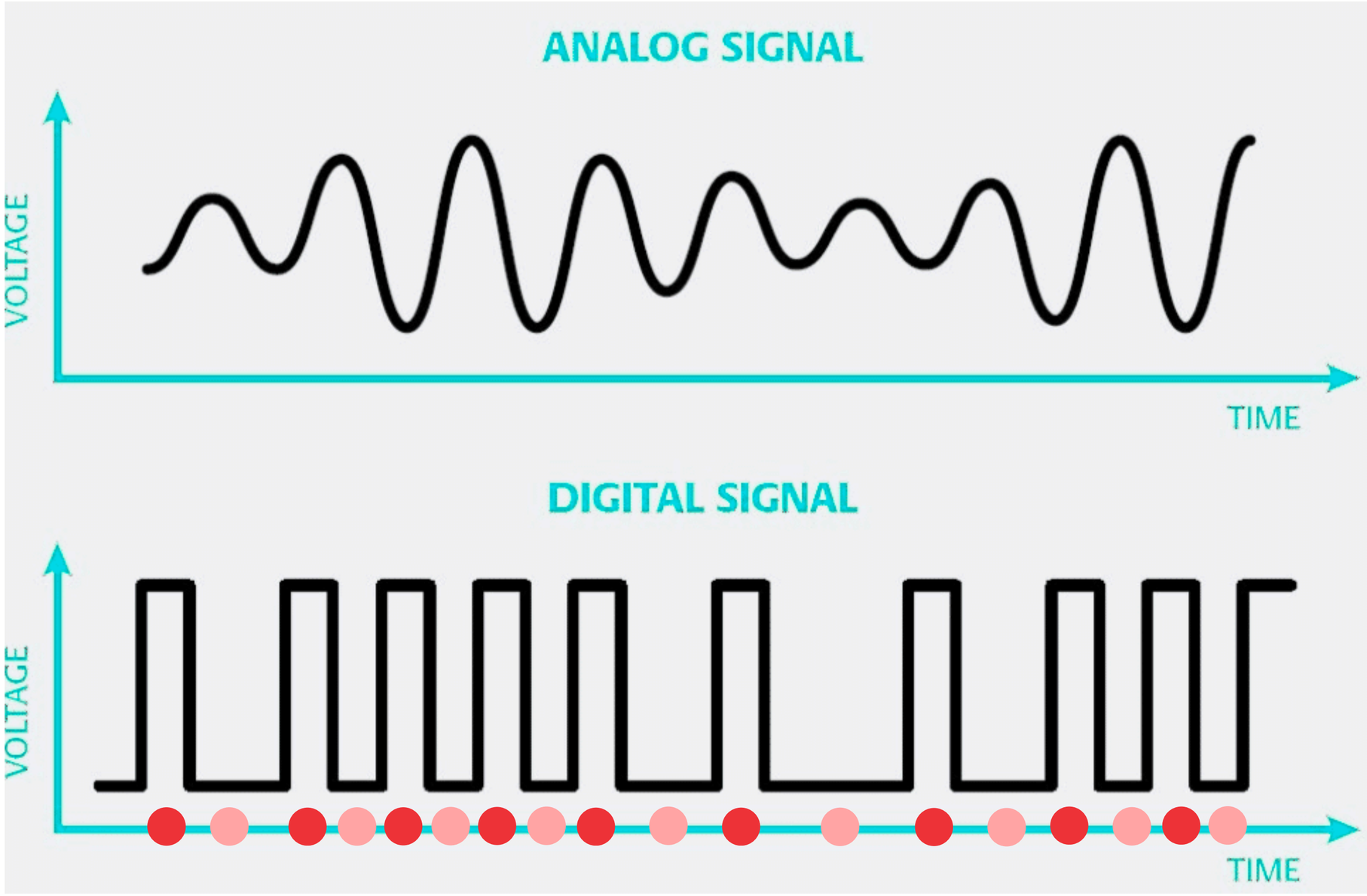

Vss
R2
D1
+5V
MICRO
Q1
c
R1
b
OUTPUT
e
LEDnique.com

DIGITAL ELECTRONICS

Digital electronics is a branch of electronics that deals with processing, storing, and transmitting information using discrete (step-by-step) signals, rather than continuous signals.




ANALOG SIGNAL
VOLTAGE
TIME
DIGITAL SIGNAL
VOLTAGE
TIME


ANALOG SIGNAL
VOLTAGE
TIME
DIGITAL SIGNAL
VOLTAGE
TIME

MICRO-CONTROLLERS

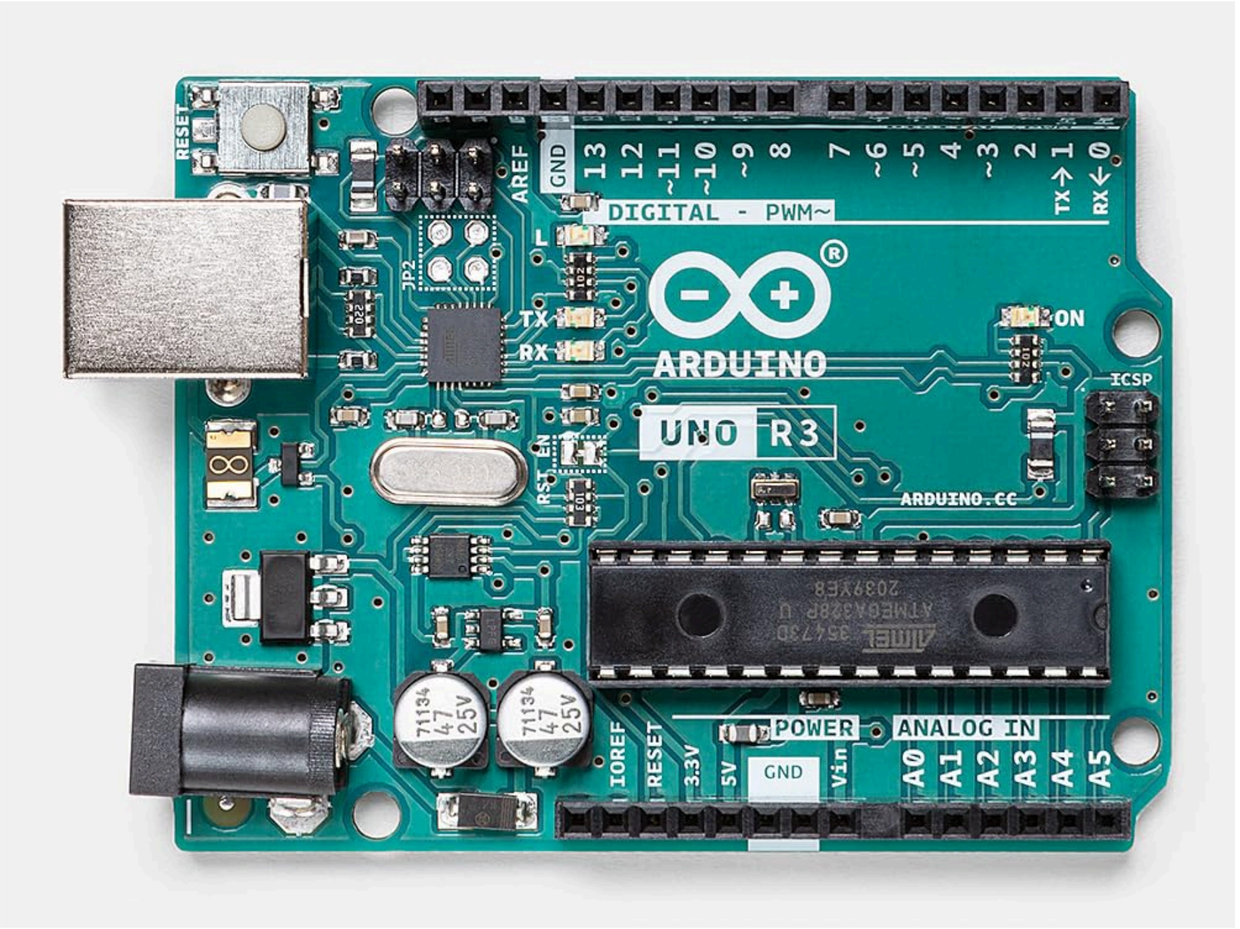
A microcontroller is a single-chip device that acts as a bridge, allowing you to control physical hardware using software.

OR

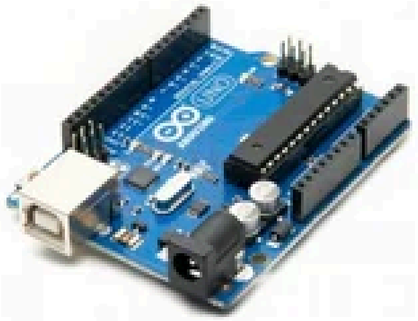
A microcontroller is a single-chip device that takes in input signals from sensors, processes that data based on its program, and uses it to control output devices.

ARDUINO

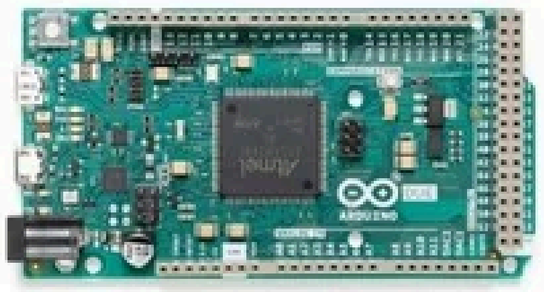
Arduino is an open-source electronics platform that gives you the complete ecosystem—both the physical hardware circuit board and the software application—to easily write code and control electronic devices.



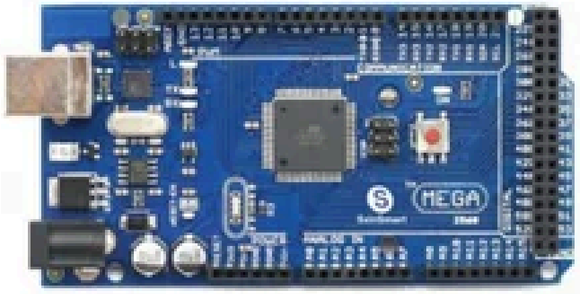
[a] Arduino Uno (R3)



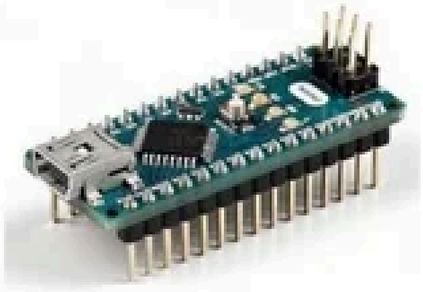
[b] Arduino Due



[c] Arduino Mega



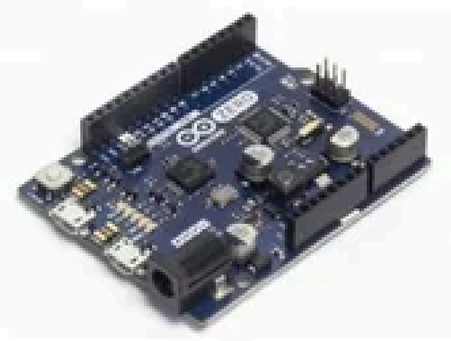
[d] Arduino Nano



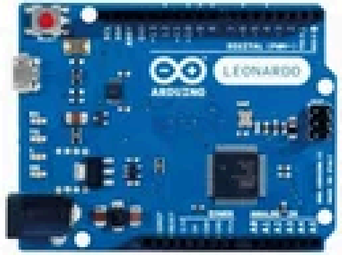
[e] Arduino Esplora

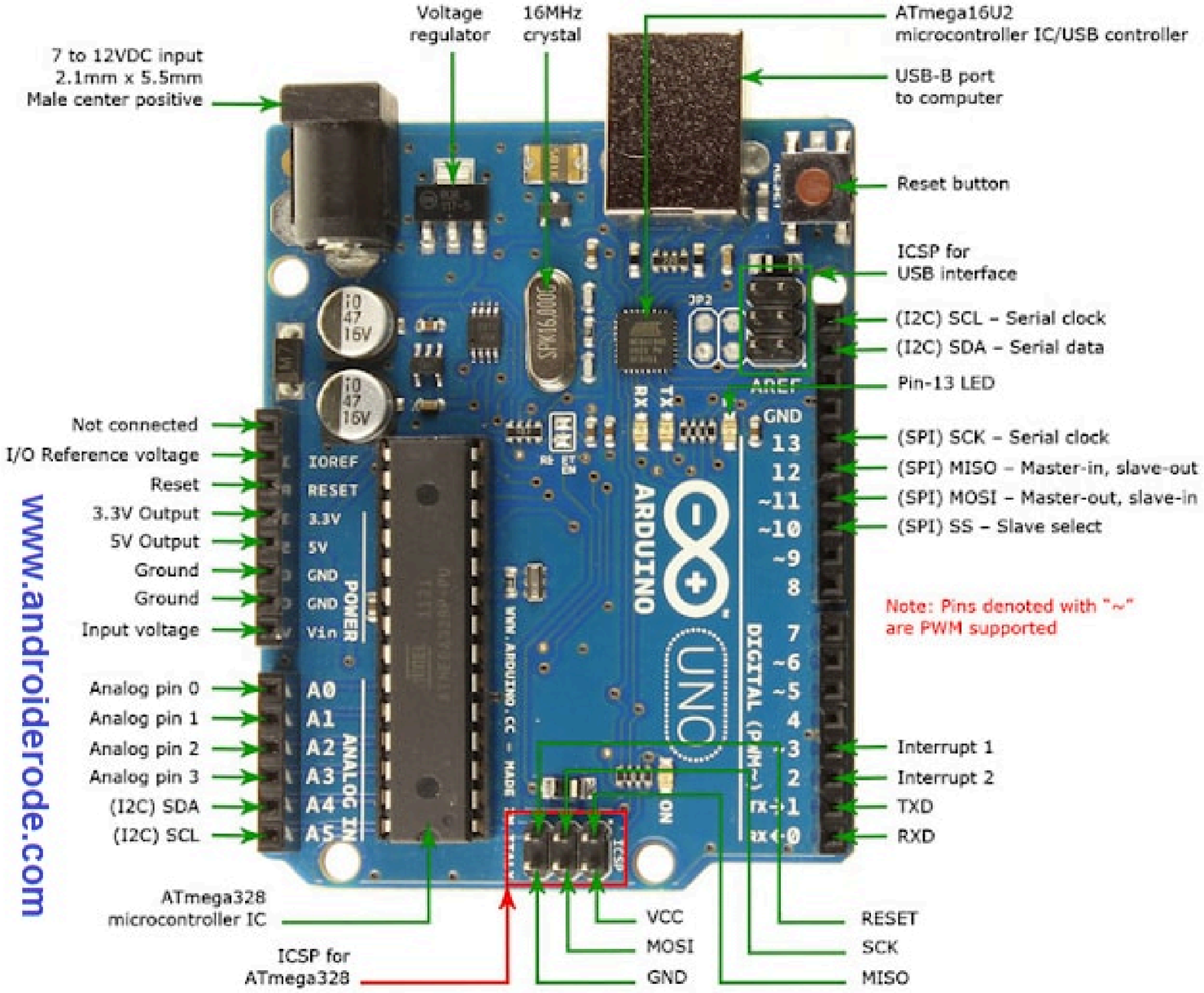


[f] Arduino Zero



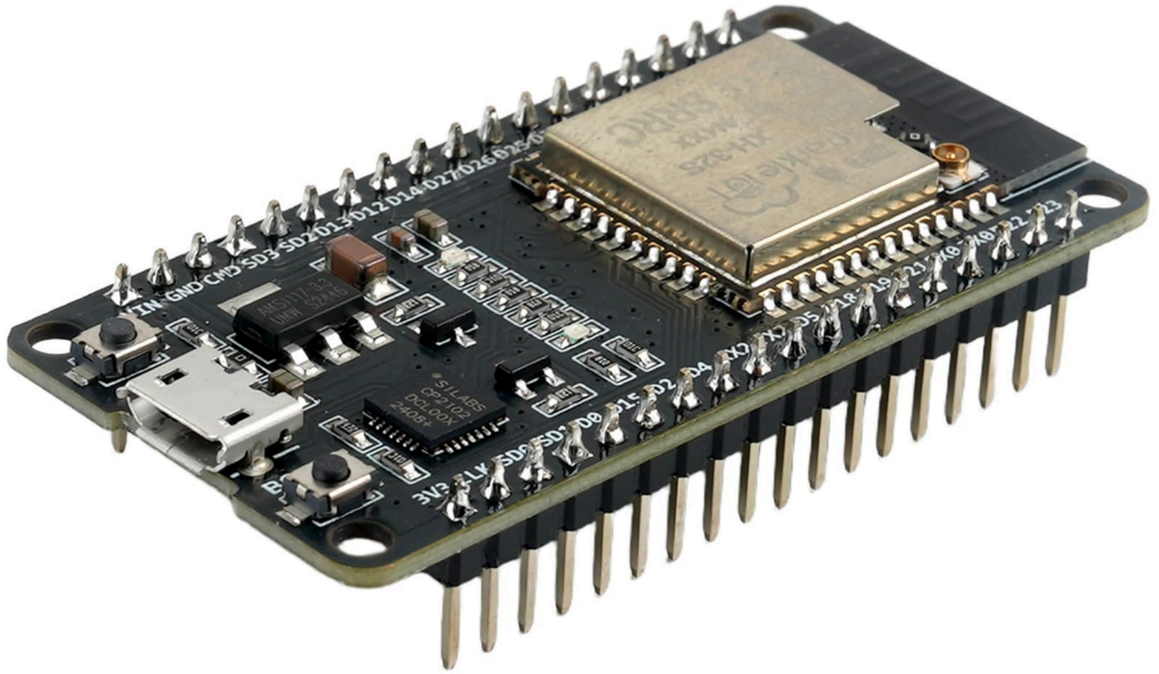
[g] Arduino Leonardo Board




Voltage regulator
16MHz crystal
ATmega16U2 microcontroller IC/USB controller
7 to 12VDC input 2.1mm x 5.5mm Male center positive
USB-B port to computer
Reset button
ICSP for USB interface
(I2C) SCL – Serial clock
(I2C) SDA – Serial data
Pin-13 LED
Not connected
I/O Reference voltage
Reset
3.3V Output
5V Output
Ground
Ground
Input voltage
(SPI) SCK – Serial clock
(SPI) MISO – Master-in, slave-out
(SPI) MOSI – Master-out, slave-in
(SPI) SS – Slave select
Note: Pins denoted with "~" are PWM supported
Analog pin 0
Analog pin 1
Analog pin 2
Analog pin 3
(I2C) SDA
(I2C) SCL
Interrupt 1
Interrupt 2
TXD
RXD
ATmega328 microcontroller IC
ICSP for ATmega328
VCC
MOSI
GND
RESET
SCK
MISO
IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
ARDUINO
UNO
DIGITAL (PWM~)
GND
13
12
~11
~10
~9
8
7
~6
~5
4
3
2
TX→1
RX←0
ICSP
ICSP2
ON
www.androiderode.com

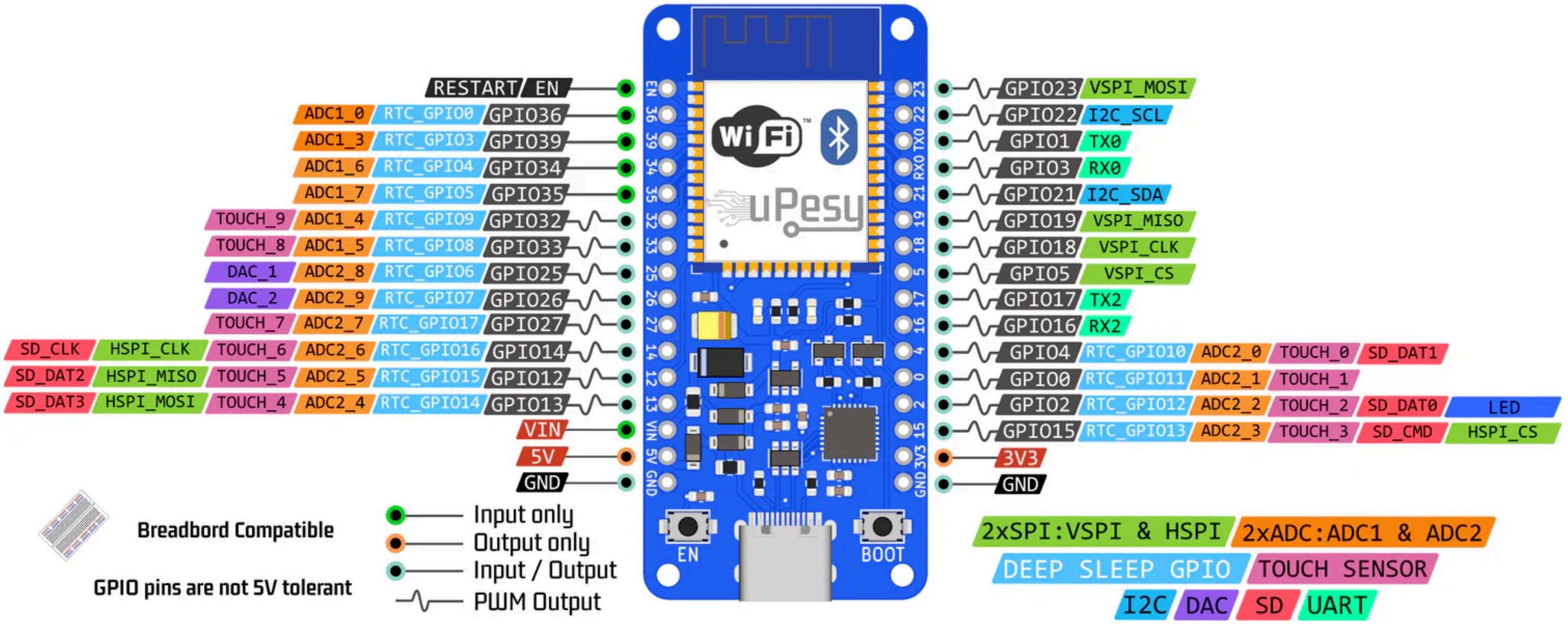
ESP32

The ESP32 is a low-cost, low-power system-on-a-chip (SoC) microcontroller that features integrated Wi-Fi and dual-mode Bluetooth connectivity.



Developed by Espressif Systems, it is explicitly engineered to act as the central brain for Internet of Things (IoT) projects, allowing your software to not only control local physical hardware but also interact with servers and services across the internet.

ESP32 Wroom DevKit Rev2 Full Pinout



Processor: Tensilica Xtensa Dual-Core 32-bit LX6 microprocessor operating at adjustable speeds from 80 MHz up to 240 MHz.
Memory Capacity: 520 KB of internal SRAM for temporary data and typically 4 MB of external Flash memory to store your software code permanently.

Operating Voltage: Built entirely on a 3.3V logic level, meaning all input and output signals must stay within 0V to 3.3V to avoid damaging the pins.

General Purpose Input/Output (GPIO): Breaks out 30 to 32 usable pins on standard development boards, with each output pin able to safely drive a continuous current of up to 12 mA.

Input-Only Pins: GPIO 34, 35, 36 (VP), and 39 (VN) do not have internal software-controlled pull-up or pull-down resistors and can only be used to read incoming signals, not control outputs.

Analog Input Channels (ADC): Features 18 independent channels of high-resolution 12-bit Analog-to-Digital Conversion, translating raw input voltages into a digital scale from 0 to 4095.

Wi-Fi Constraint on ADC: The 10 analog channels tied to the ADC2 module become completely unavailable when the built-in Wi-Fi is actively transmitting data, requiring you to use the 8 channels on ADC1 (GPIOs 32 to 39) for continuous sensor monitoring.

Analog Output Channels (DAC): Offers 2 channels of true 8-bit Digital-to-Analog Conversion on GPIO 25 and 26, allowing the software to output smooth, variable voltage waveforms rather than rough simulated signals.

Pulse Width Modulation (PWM): Provides up to 16 independent PWM channels with configurable resolution up to 16-bits, which can be internally routed through software to almost any output-capable GPIO pin.

Capacitive Touch Pins: Includes 10 specialized pins (GPIOs 0, 2, 4, 12, 13, 14, 27, 32, 33) with built-in touch-sensing hardware that can detect a human finger through protective glass or plastic pads.

Wireless Networking: Features integrated 2.4 GHz Wi-Fi (802.11 b/g/n) supporting advanced web security layers alongside dual-mode Bluetooth (Classic Bluetooth for data streaming and Bluetooth Low Energy for ultra-low-power sensing).

Serial Communication (UART): Contains 3 independent hardware serial ports (UART0, UART1, and UART2) to simultaneously communicate with your computer and external modules like GPS trackers or cellular modems.

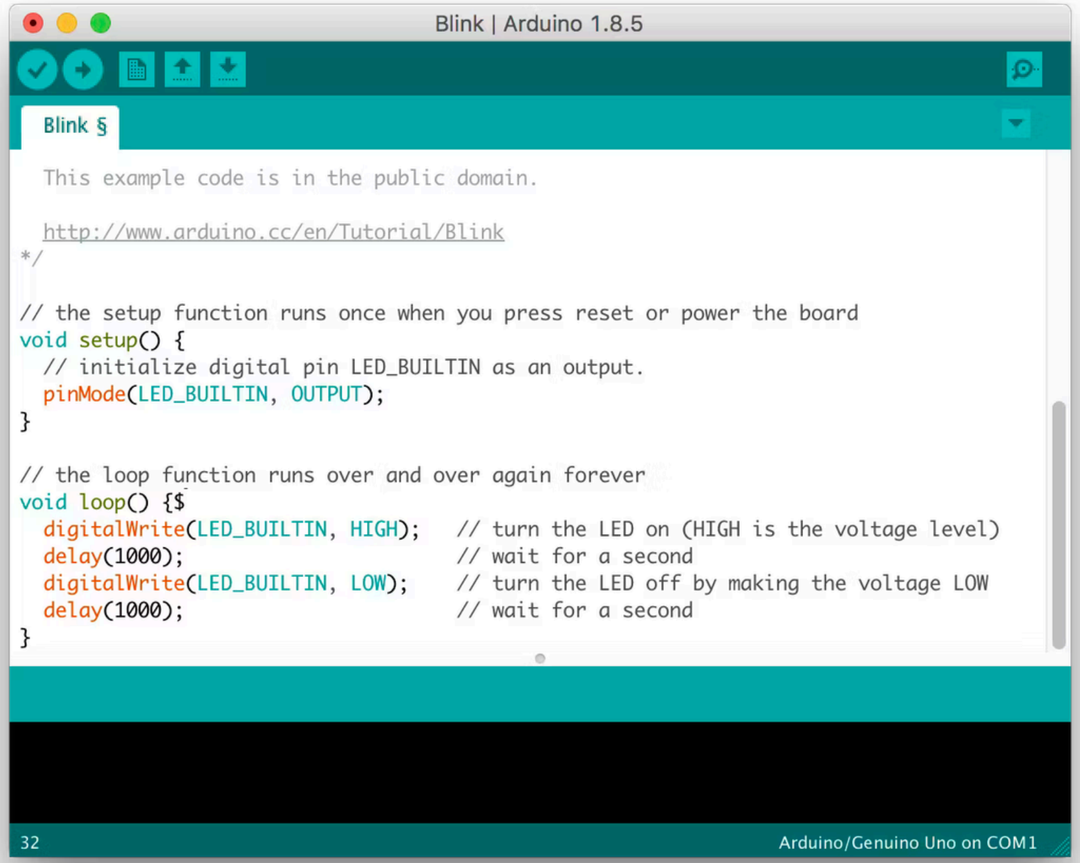
I2C Protocol Support: Includes 2 hardware Inter-Integrated Circuit interfaces that can be mapped to almost any pins to run multiple sensor modules using only 2 shared signal wires.

SPI Protocol Support: Features 3 high-speed Serial Peripheral Interface buses (SPI, HSPI, and VSPI) designed to handle intensive data communication with hardware like SD cards and graphic screens.

Hardware Security Acceleration: Integrates dedicated silicon cryptographic engines to compute internet encryption standards like AES, SHA, and RSA directly in hardware, freeing up the main processor cores to execute your core control logic without delay.

ARDUINO IDE

The Arduino IDE (Integrated Development Environment) is the official software application that acts as the control station where you write, test, compile, and upload your software code into a microcontroller board.

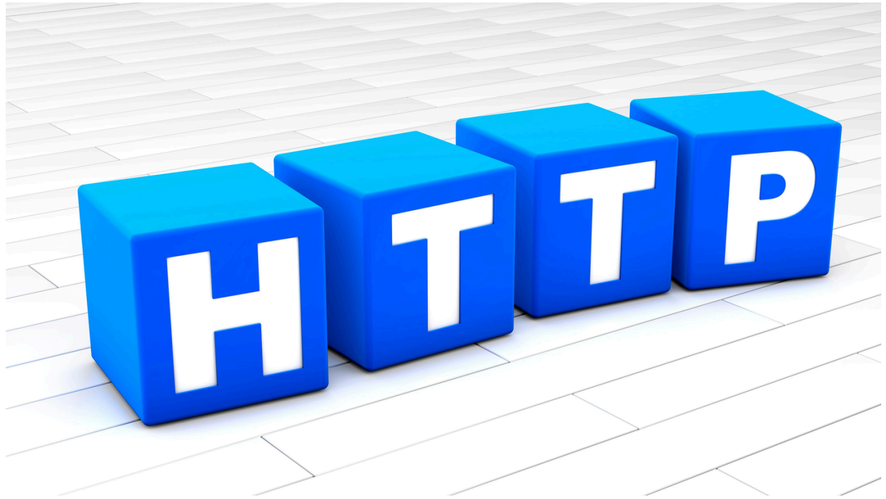


HTTP Protocol

HTTP (Hypertext Transfer Protocol) is the foundational, text-based protocol used to transfer data across the internet. It operates on a classic Client-Server model, where a client (such as your laptop browser or smartphone) sends an outbound request, and a server (such as an ESP32 or a cloud database) processes that request and sends back a response.

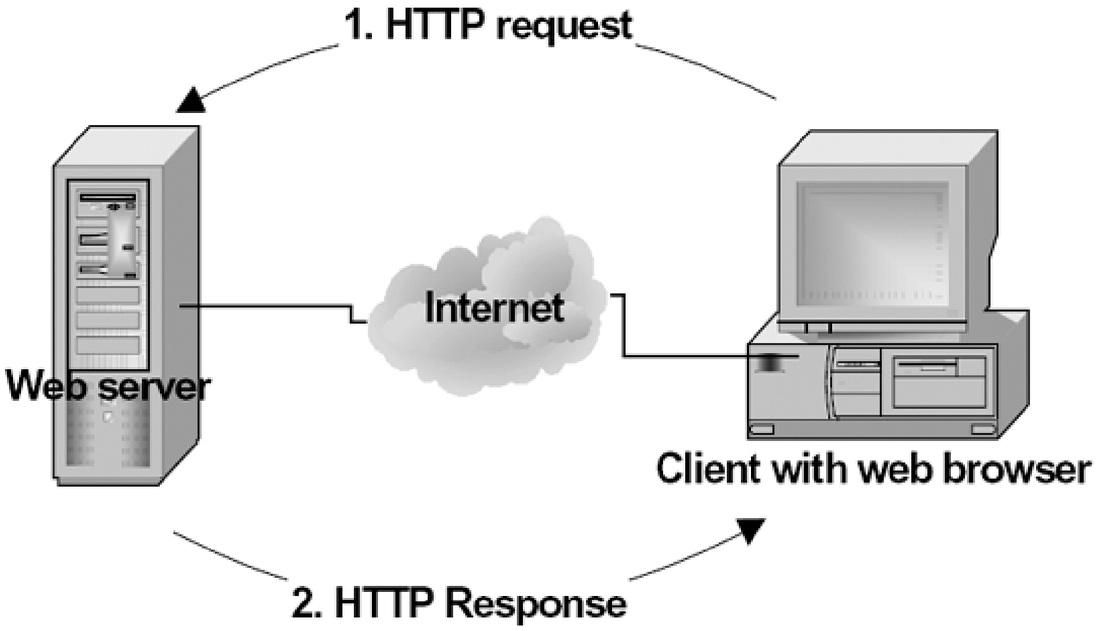
Advantages of HTTP

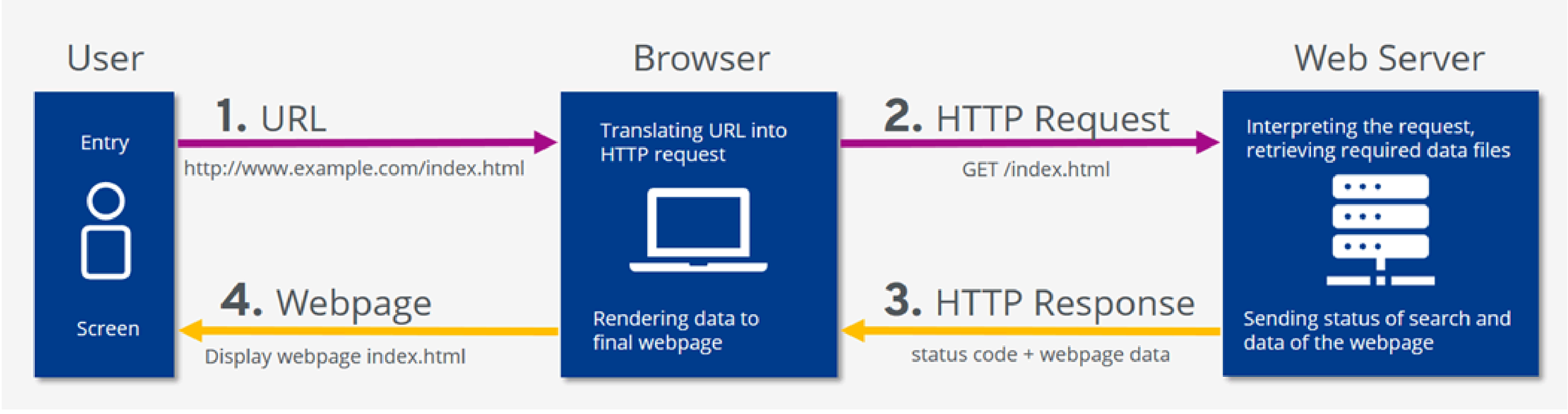
- Universal Compatibility: It is the standard language of the web. Every browser, smartphone, cloud database, and modern programming language supports it natively out of the box.
- Simple and Human-Readable: Because HTTP uses clear text headers and standard actions (like GET and POST), it is incredibly easy to read, write, and debug using simple tools.
- Firewall Friendly: HTTP typically communicates on standard network ports (Port 80 for unencrypted text and Port 443 for secure SSL). Routers and corporate firewalls usually keep these ports open by default, preventing connection blocks.
- Stateless Reliability: Each request is entirely independent and self-contained. If one transaction fails due to a temporary signal drop, it does not crash or corrupt the history of the next transmission.



Disadvantages of HTTP

- High Packet Overhead: HTTP is a heavy protocol. To send a single, tiny numeric character command (like 1 or 0 to control an LED), the chip must pack it inside hundreds of bytes of mandatory text headers, cookies, and metadata.
- Inefficient Polling (Latency): To receive commands from the web, an ESP32 client must repeatedly open a connection, download a file, and tear down the connection over and over again. This continuous cycling drains processing power and causes a delay in action response.
- High Power Consumption: Because opening and closing secure TCP/IP connection handshakes requires significant processor execution time and radio uptime, it drains battery power quickly compared to lightweight streaming alternatives.
- One-Way Initiation: The communication must always be started by the client. An external web server cannot directly reach through a home router to push a sudden command down to an ESP32 client without a complex setup; it must wait for the chip to check in first.


1. HTTP request
Internet
Web server
Client with web browser
2. HTTP Response


User
Browser
Web Server
Entry
Screen
1. URL
http://www.example.com/index.html
Translating URL into HTTP request
Rendering data to final webpage
2. HTTP Request
GET /index.html
Interpreting the request, retrieving required data files
Sending status of search and data of the webpage
3. HTTP Response
status code + webpage data
4. Webpage
Display webpage index.html

MQTT Protocol

MQTT (Message Queuing Telemetry Transport) is a lightweight, publish-subscribe network protocol designed specifically for resource-constrained microcontrollers and low-bandwidth, high-latency networks.
Instead of clients talking directly to a server, MQTT uses a central Broker. Devices connect to this broker and either Publish data to a specific text pathway (called a Topic) or Subscribe to a topic to listen for updates.

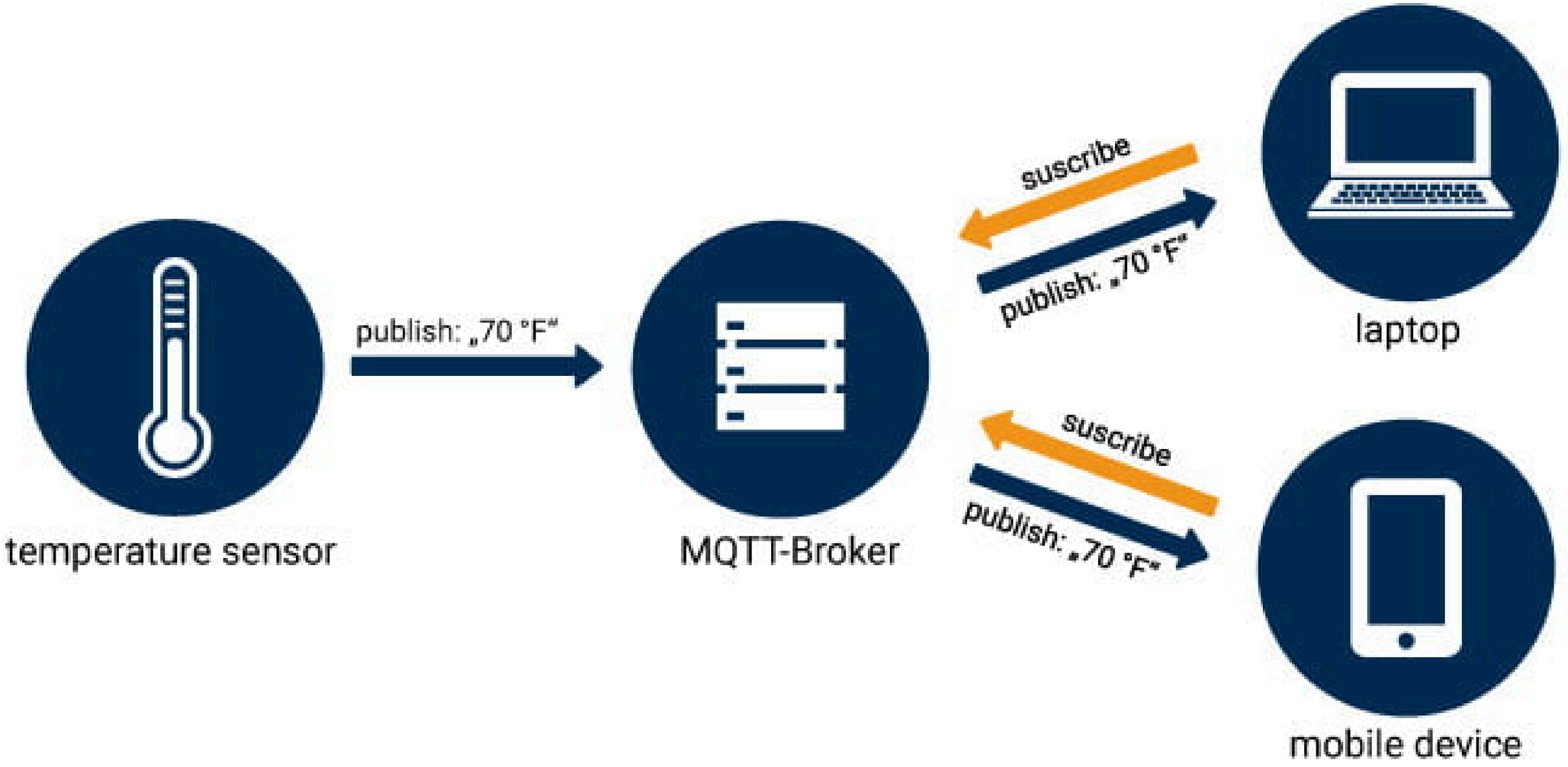
Advantages of MQTT

- Ultra-Lightweight and Fast: MQTT has a tiny packet overhead (its fixed header is only 2 bytes). This allows the ESP32 to transmit or receive commands with minimal network data, making operations feel instantaneous.
- Real-Time Event Pushing: Because it uses an asynchronous "push" model, you do not need to repeatedly query a file to check for updates. The broker actively pushes new data to the subscriber the millisecond it arrives.
- Low Power Consumption: Keeping a thin TCP/IP socket idle consumes drastically less battery power than continuously opening, downloading, and destroying heavy HTTP connection stacks.
- Bi-directional Through Firewalls: Once an ESP32 makes an initial outbound connection to a cloud broker, that pipeline stays open. This allows an external phone or computer to send commands down to the chip instantly, even if the hardware is buried deep behind a private home router firewall.



Disadvantages of MQTT

- Requires a Central Broker: MQTT cannot function in a vacuum. You must deploy, maintain, or pay for a dedicated central broker server (like Mosquitto, HiveMQ, or EMQX) to handle the message traffic distribution.
- No Built-In Data History: Standard MQTT brokers are strictly data routers, not databases. They deliver a message to active subscribers and immediately delete it from memory. If you want to log data over time, you must attach a separate storage engine to the pipeline.
- Increased System Complexity: The asynchronous publish/subscribe model requires managing unique topic strings, subscription lists, and state variables, which is a steeper learning curve for beginners than basic web page fetching.
- Potential for Topic Crossovers: In a classroom or development setting, if multiple students accidentally use the exact same public broker and topic string, their devices will intercept each other's commands, causing random device actions.


suscribe
publish: „70 °F“
laptop
publish: „70 °F“
temperature sensor
MQTT-Broker
suscribe
publish: „70 °F“
mobile device